

Sri Lanka Institute of Information Technology



Data Warehousing and Business Intelligence (IT3021) Assignment 1

IT Number	Name	Email	Batch	Center
IT23343320	Senadeera D.M.K.K	IT23343320@my.sliit.lk	Y3.S2.WE.DS.02.02	Malabe

Contents

Task 1: Data Set Selection	3
1.1 Overview	3
1.2 Dataset Description	3
1.3 Justification for Selection	5
1.4 ER Diagram	7
Task 2: Preparation of data sources.....	8
2.1 Data Source Identification & Organization.....	8
2.1.1 Source 1 – SQL Server Database (Hospital_Source_DB).....	8
2.1.2 Source 2 – Excel / CSV Files	9
2.1.3 Source 3 – Plain Text File (billing_detail.txt).....	10
Task 3: Solution architecture.....	11
3.1 Architecture Diagram	11
3.2 Component Summary & Detailed Description.....	12
Task 4: Data warehouse design & development	14
4.1 Fact Table Grain Definition	14
4.2 Slowly Changing Dimensions (SCDs).....	14
4.3 Confirmed Dimensions	15
4.4 Snowflakes Schema Relational Diagram	18
4.5 Design Assumptions	18
Task 5: ETL Development	19
5.1 Extraction Strategy & Source Diversity	19
5.3 Key Transformation Logic	38
5.4 Error Handling & Validation.....	42
Task 6: ETL Development – Accumulating Fact Tables	43

Task 1: Data Set Selection

Dataset Link: <https://www.kaggle.com/datasets/shalakagangurde/hospital-hmis-dataset-for-healthcare-analytics>

1.1 Overview

The dataset models how a real hospital captures, stores and manages information related to patient admissions, clinical events, financial billing, insurance, ward management, and staff operations. It is a relational, normalized dataset that spans multiple years of synthetic hospital data from 2020 to 2025, giving it a sufficient time range for building a data warehouse with meaningful historical trends.

1.2 Dataset Description

Table Name	Description	Key Attributes
patient	Patient demographics	patient_id, gender, date_of_birth, blood_group, city, contact_number
department	Hospital departments and floors	department_id, department_name, department_type, floor_number, status
ward	Wards within departments	ward_id, ward_name, ward_type, total_beds, department_id
disease	Disease catalogue	disease_id, disease_name, disease_category
Insurance_provider	Insurance companies offering coverage	insurance_provider_id, provider_name, provider_type, contact_details, coverage_limit
admission	Patient admission events (central transactional table)	admission_id, admission_date, discharge_date, admission_type, admission_status,

		patient_id, department_id, ward_id, disease_id
billing	Overall billing record per admission	bill_id, bill_date, total_amount, insurance_covered_amount, patient_payable_amount, payment_status, payment_mode, admission_id
billing_detail	Line-item breakdown of each bill	billing_detail_id, charge_type, reference_id, amount, bill_id
patient_insurance	Insurance policies held by patients	patient_insurance_id, policy_number, coverage_percentage, policy_start_date, policy_end_date, patient_id, insurance_provider_id
bed	Details of beds within wards and their availability status	bed_id, bed_number bed_status, ward_id
doctor	Details of doctors, including specialization and experience	doctor_id, employee_id, specialization, qualification, experience_years
drug	Catalogue of medicines available in the hospital	drug_id, drug_name, brand_name, drug_category, unit_cost, manufacturer_id
drug_inventory	Pharmacy inventory levels, stock status, and reorder details	inventory_id, current_stock, reorder_level, inventory_status, last_restock_date, drug_id
Employee	Details of hospital staff, including roles and employment types	employee_id, employee_name, gender, role, employment_type, date_of_joining, department_id

prescription	Records of medications prescribed to admitted patients	prescription_id, dosage, frequency, duration_days, admission_id, drug_id
staff_assignment	Details of staff shifts and their assigned wards	assignment_id, employee_id, ward_id, shift

1.3 Justification for Selection

The **Hospital Management System (HMS)** dataset was selected based on the following key reasons:

- **Genuine OLTP Structure:** It is a highly normalized relational dataset, providing a realistic challenge for designing a Star Schema from scratch.
- **Clear Dimensional Hierarchy:** The schema contains natural dimensions (Patient, Ward, Disease) surrounding a central transactional entity (Admission).
- **Significant Temporal Depth:** Spanning from **2020 to 2025**, it enables robust time-series analysis and Year-over-Year (YoY) growth tracking.
- **SCD Implementation Ready:** The dataset naturally supports Slowly Changing Dimensions. Attributes like Age, City, and Contact Number are implemented using SCD Type 1 (overwrite), while Insurance Name utilizes SCD Type 2 (historical tracking).
- **Accumulating Snapshot Potential:** The lifecycle from Admission to Discharge is perfect for demonstrating an **Accumulating Snapshot Fact Table** to calculate the Average Length of Stay (ALOS).
- **Financial Analytics:** The billing sub-domain provides essential measures for fact table design and introduces **Degenerate Dimensions** like Bill Number.
- **Novelty:** The dataset is original and not based on common samples like AdventureWorks, ensuring the uniqueness of the project.

Design Assumptions

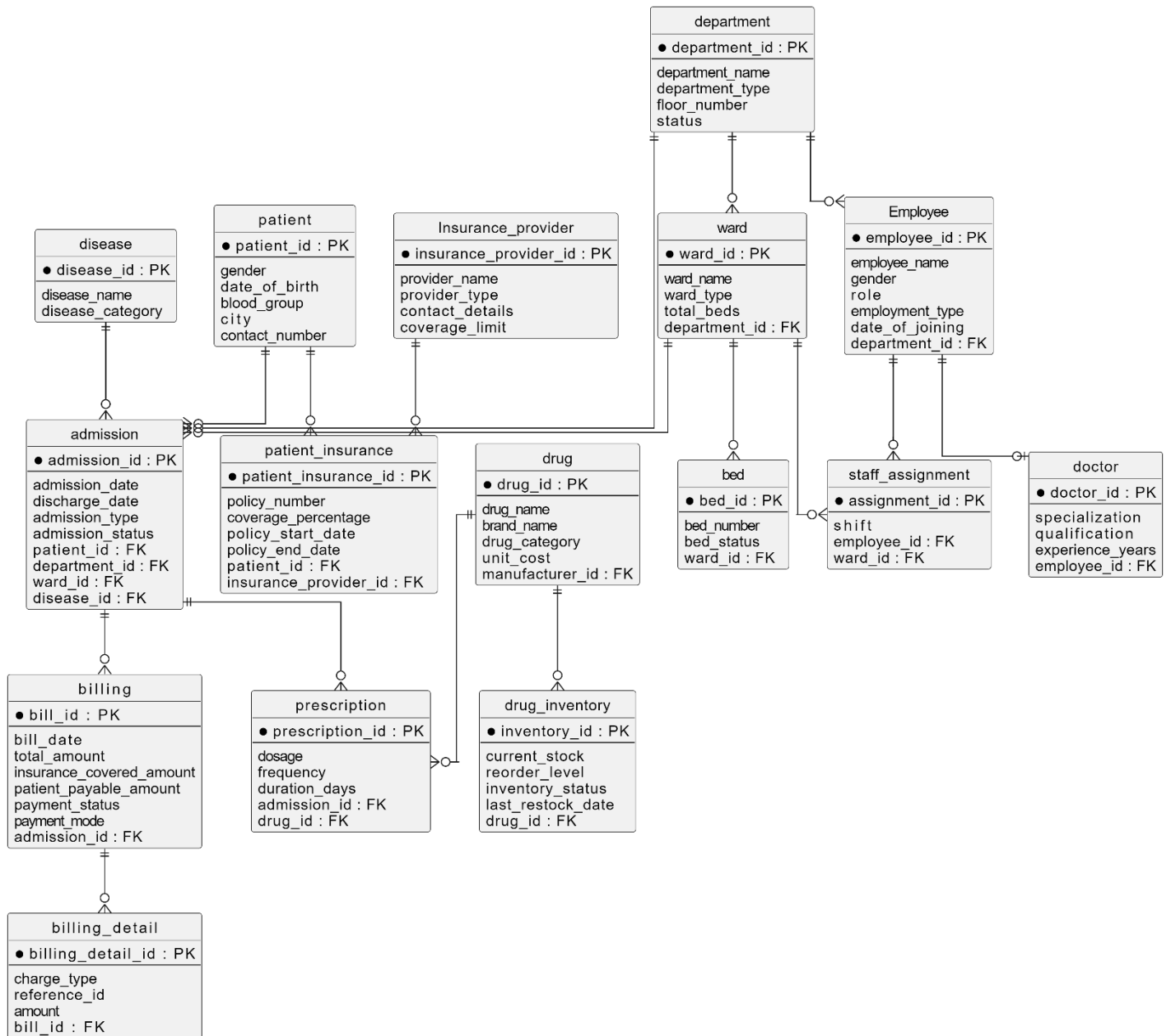
The following assumptions were made to ensure a consistent Data Warehouse design:

- **SCD Implementation Logic:** It is assumed that tracking historical changes for a patient's Insurance Name is critical for long-term financial analytics (handled via SCD Type 2). Conversely, demographic updates such as Age, City, and Contact Number require only the most current data for analysis and are therefore handled via SCD Type 1.

- **Currency Uniformity:** All billing amounts are assumed to be in a single consistent currency (LKR).
- **Unique Identification:** The admission_id from the source system is used as the unique natural key to identify transactions in the Fact table.

1.4 ER Diagram

ER diagram below illustrates the nine selected tables.



Task 2: Preparation of data sources

2.1 Data Source Identification & Organization

The three source types used in this project are:

- Source 1 - SQL Server Database (Hospital_Source_DB): twelve tables loaded into a SQL Server database to simulate the hospital's core operational system.
- Source 2 - Excel / CSV Files: three tables kept as flat files (Excel/CSV format) to simulate data feeds from external or administrative systems.
- Source 3 - Plain Text File (txt): one large transactional file kept in delimited text format to simulate a bulk export from a billing sub-system.

2.1.1 Source 1 – SQL Server Database (Hospital_Source_DB)

Five tables were imported into a SQL Server database named Hospital_Source_DB. These tables represent the core clinical and structural data of the hospital -the kind of information that would typically be maintained directly inside a hospital's operational database system. Loading these into SQL Server allows us to use a database connection as one of the ETL source types in SSIS.

Table	Information Available	Rows Count
Admission	Central transactional table. Records each patient admission event with admission/discharge dates, type, status, and foreign keys to patient, department, ward and disease.	45000
Billing	One billing record per admission. Stores total amount, insurance covered amount, patient payable amount, payment status and payment mode	45000
Department	Lists hospital departments with name, type, floor number and operational status.	11
Disease	Disease catalogue with disease name and category (e.g. Cardiovascular, Respiratory).	20
Ward	Ward details including name, type, total bed count, and the department it belongs to.	27
Bed	Details of individual hospital beds. Stores the specific bed number and its current availability status within a ward.	415

Doctor	Specific professional details for medical doctors. Stores the doctor's specialization, qualifications, and years of experience.	98
Drug	A catalog of available medicines in the hospital. Stores drug name, brand name, drug category, unit cost, and links to its manufacturer.	250
Drug_inventory	Tracks the hospital's pharmacy stock levels. Stores current stock count, reorder levels, inventory status, and the date of the last restock.	250
Employee	General information about all hospital staff. Stores employee name, gender, role, employment type, date of joining, and their assigned department.	500
Prescription	Records of medications prescribed to patients. Stores dosage, frequency, duration in days, and links to the admission and the specific drug.	73109
Staff Assignment	Tracks the working schedules of hospital staff. Stores shift details and assigns specific employees to specific wards.	207

2.1.2 Source 2 – Excel / CSV Files

Three tables were kept as Excel-compatible CSV files. These represent patient-facing and administrative data that might realistically be maintained in spreadsheets or exported from an external patient registration system or insurance portal. Using flat files as a second source type satisfies the multi-source requirement and introduces a different kind of ETL source connection in SSIS (Flat File Connection vs OLE DB Connection).

Table	Information Available	Rows Count
patient.csv (Excel/CSV)	Patient demographic records: gender, date of birth, blood group, city and contact number. Used to build the Patient dimension.	30000
insurance_provider.csv (Excel/CSV)	Details of insurance companies: provider name, type, contact details and coverage limit.	50

patient_insurance.csv (Excel/CSV)	Patient-to-insurance policy mapping: policy number, coverage percentage, start and end dates	21617
---	--	-------

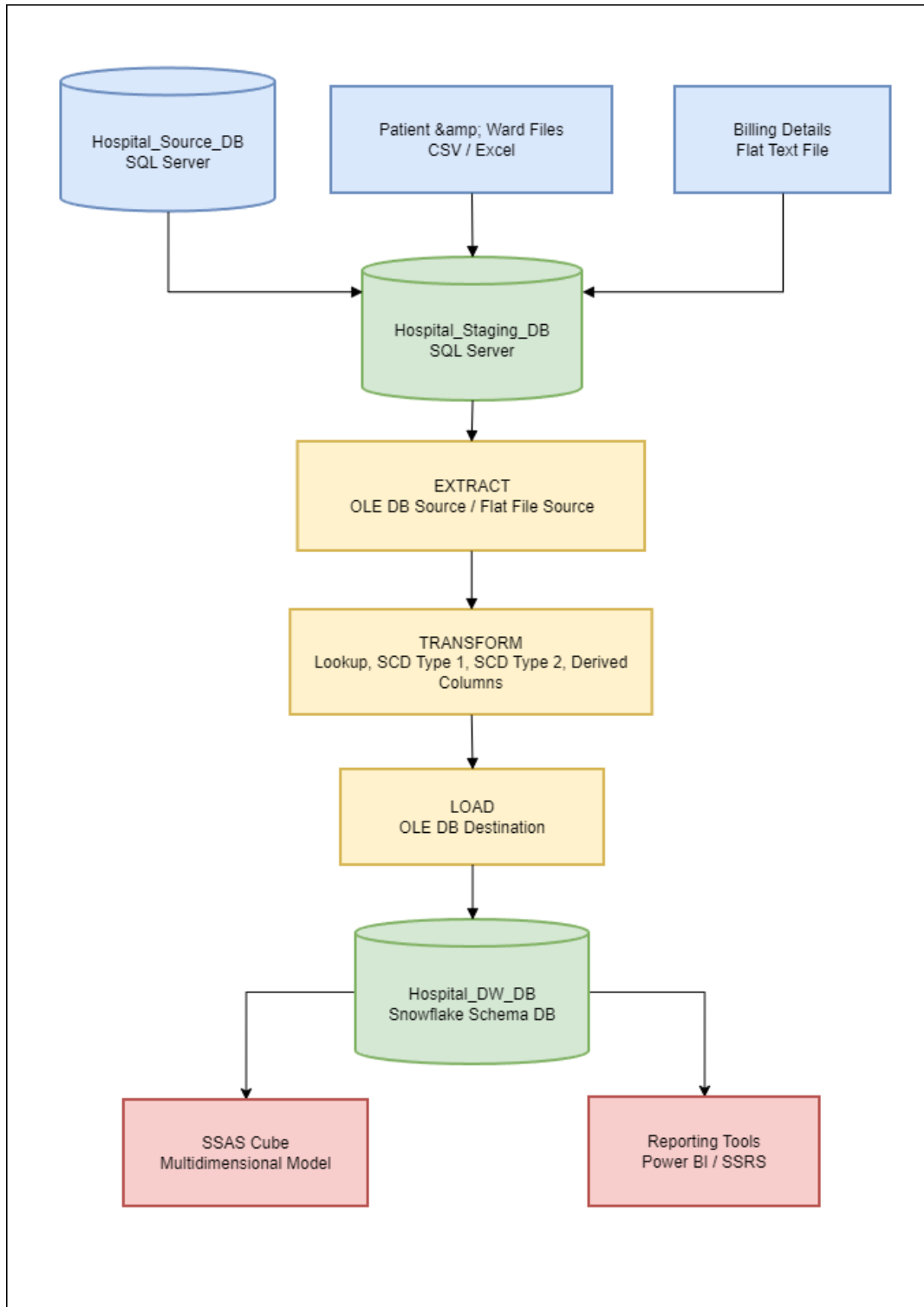
2.1.3 Source 3 – Plain Text File (billing_detail.txt)

The billing_detail table was exported and saved as a plain text file (billing_detail.txt) with comma-separated values. This file represents the most granular level of financial data in the dataset , the individual charge line items that make up each bill. Keeping this as a text file introduces a third source type and simulates the kind of bulk data dump that might be received from a separate billing sub-system in a real hospital environment.

Table	Information Available	Approx. Rows
billing_detail.txt (Plain Text)	Line-item breakdown of each bill: charge type, reference ID and amount, linked to the parent bill by bill_id. This is the most granular financial table in the dataset.	112402

Task 3: Solution architecture

3.1 Architecture Diagram



3.2 Component Summary & Detailed Description

The architecture of the Hospital HMIS DW/BI solution is comprised of five distinct layers, each serving a specific functional role in the data lifecycle—from raw ingestion to analytical insights.

Layer 1: Data Sources (Heterogeneous OLTP Input)

The foundation of the system consists of three diverse data source types. This multi-source configuration was intentionally designed to simulate a fragmented healthcare environment.

- SQL Server (Hospital_Source_DB): Acts as the core clinical system, housing relational tables for Admissions, Wards, and Departments etc...
- Flat Files (CSV/Excel): Represent external administrative feeds for Patient demographics and Insurance provider details.
- Plain Text (.txt): A high-volume delimited file used specifically for granular billing line items, simulating a legacy billing sub-system export.

Rationale: This variety ensures the ETL engine is capable of handling different connection managers and data formats, a key requirement for modern data integration.

Layer 2: Staging Area (Hospital_Staging_DB)

The Staging Area is a dedicated SQL Server database that serves as a "landing zone" for raw data. Before any transformation occurs, data is moved here to decouple the source systems from the Data Warehouse.

- Role: It provides a safe environment to perform data quality checks and cleansing without placing an ongoing load on the operational hospital databases.
- Structure: It contains seventeen Stg_ tables that mirror the source schema, allowing for a "truncate and load" strategy that simplifies ETL restarting in case of failure.

Layer 3: ETL Engine (SSIS Integration)

The ETL logic is orchestrated by three primary SQL Server Integration Services (SSIS) packages. This layer is the "brain" of the solution, where complex business logic is applied.

- Transformation Logic: The engine handles Data Conversion to ensure type compatibility, uses Lookup Transformations to resolve surrogate keys, and implements Derived Columns to calculate metrics like StayDuration.
- SCD Type 1 and Type 2 Implementation: Specifically for the Dim_Patient table, the SSIS SCD Wizard is used to track historical changes in patient insurance, changing attributes in patient City and Contact Number ensuring that analytical reports reflect the patient's status at the time of the event.

Layer 4: Data Warehouse (Hospital_DW)

The Hospital_DW_DB is the central analytical repository, modeled using a Snowflake Schema to optimize query performance.

- **Dimensions:** Contains conformed dimensions such as Dim_Date, Dim_Bed, Dim_Department, Dim_Disease, Dim_Drug, DimEmployee, Dim_Ward, and Dim_Patient. These provide the context (Who, Where, When) for analysis.
- **Fact Table:** Fact_Admission_Billing acts as the center of the star, storing quantitative measures. It follows an Accumulating Snapshot pattern, capturing the full lifecycle of a patient stay from admission to discharge.

Layer 5: BI & Reporting Layer (Analysis & Delivery)

The final layer transforms processed data into actionable intelligence for hospital administrators.

- **SSAS (Analysis Services):** A multidimensional OLAP cube was developed to enable rapid "slice-and-dice" analytical capabilities. It incorporates logical hierarchies (e.g., Date hierarchies) and predefined measures, allowing users to perform deep-dive analysis seamlessly.
- **SSRS (Reporting Services):** Utilizing modern reporting tools, the multidimensional data is visualized to support both daily operations and strategic decision-making:
 - **Operational Analytics:** Interactive visualizations, such as temporal admission trends and "Ward-wise Patient Distribution," help administrators manage hospital staffing, monitor capacity, and optimize resource allocation.
 - **Financial Summaries:** Comprehensive financial dashboards analyze total revenue generation (e.g., tracking the 1.68bn total amount). They specifically compare InsuranceCoveredAmount versus PatientPayableAmount across different departments, providing clear visibility into the hospital's revenue streams.
 - **Detailed Patient Profiling:** Granular reports detailing patient demographics (Age, Gender, Blood Group, City) tied to their admission and billing counts, enabling targeted regional health analytics.
 - **Efficiency Tracking:** By leveraging derived metrics such as txn_process_time_hours, the reporting layer can generate turnaround analysis to identify administrative bottlenecks in the admission and discharge processes.

Task 4: Data warehouse design & development

The proposed Data Warehouse for the Hospital Management Information System (HMIS) follows a **Snowflakes Schema** architecture. This design is optimized for high-performance analytical queries and simplified reporting for hospital administrators. The model consists of one central fact table and multiple dimension tables as described below:

4.1 Fact Table Grain Definition

The heart of the dimensional model is the Fact_Admission_Billing table. This table captures every instance of a patient being admitted to the hospital, representing the primary business process of the HMIS.

- **Grain:** The grain of this fact table is defined as "**One row per hospital admission transaction,**" ensuring the highest level of detail for analytical reporting.
- **Measures (Quantitative Data):**
 - **TotalAmount:** Represents the total billing amount for the specific admission.
 - **StayDuration:** Measures the total number of days the patient remained in the hospital from admission to discharge.
 - **txn_process_time_hours:** A calculated metric representing the total time taken (in hours) to process the transaction from creation to completion.
- **Foreign Keys (Surrogate Keys):**
 - The table contains several surrogate keys, including **PatientKey**, **WardKey**, **DiseaseKey**, and **AdmissionDateKey**.
 - These keys facilitate the **Snowflakes Schema** connections to the respective dimension tables for multi-dimensional analysis.

4.2 Slowly Changing Dimensions (SCDs)

The Dim_Patient dimension is implemented using a hybrid Slowly Changing Dimension (SCD) strategy to ensure robust data integrity and historical accuracy. Based on the ETL pipeline configuration, the attributes are handled as follows:

- **Business Key:** patient_id is utilized as the primary natural key originating from the source system.
- **Fixed Attributes:** gender, date_of_birth, and blood_group. These fundamental demographic details are treated as immutable and will not change over time.

- **Changing Attributes (SCD Type 1):** age, city, and contact_number. If a patient's address, phone number, or age requires an update, the existing record is simply overwritten to maintain the most current operational information without retaining past values.
- **Historical Attributes (SCD Type 2):** insurance_name. Tracking changes in a patient's insurance provider is critical for accurate financial and billing analytics. When an insurance policy changes, a new record is created with appropriate start_date, end_date, and is_current flags to strictly preserve the historical context of the patient's coverage.

4.3 Confirmed Dimensions

The following dimensions are shared across the hospital data warehouse, ensuring consistent and integrated analysis for healthcare reporting:

1. Dim_Patient (Slowly Changing Dimension)

- **patient_key (PK):** Surrogate key for the patient dimension.
- **patient_id (NK):** Natural key derived from the source system.
- **gender:** Patient gender (Fixed Attribute).
- **date_of_birth:** Patient date of birth (Fixed Attribute).
- **blood_group:** Patient blood group (Fixed Attribute).
- **city:** Current city of residence (SCD Type 1 - Overwrite).
- **contact_number:** Current contact details (SCD Type 1 - Overwrite).
- **age:** Current age of the patient (SCD Type 1 - Overwrite).
- **insurance_name:** Combined insurance provider details (SCD Type 2 - Historical tracking).
- **start_date:** The date from which the record version is valid.
- **end_date:** The date until which the record version was valid.
- **is_current:** Flag indicating the most recent and active patient record.

2. Dim_Department

- **department_key (PK):** Surrogate key for the department dimension.
- **department_id (NK):** Natural key identifying the department from the source.
- **department_name:** The descriptive name of the medical department (e.g., Surgery).
- **department_type:** The classification or category of the department.
- **floor_number:** The physical floor location of the department within the hospital.

- **status:** Current operational status of the department.

3. Dim_Ward

- **ward_key (PK):** Surrogate key for the hospital ward.
- **ward_id (NK):** Original ward identifier from the source system.
- **ward_name:** Name of the ward (e.g., Emergency Ward 1).
- **ward_type:** Classification of the ward.
- **total_beds:** The total bed capacity of the specific ward.
- **department_key (FK):** Foreign key linking the ward to its respective medical department in Dim_Department (Snowflake hierarchy).

4. Dim_Bed

- **bed_key (PK):** Surrogate key representing an individual hospital bed.
- **bed_id (NK):** Natural key for the bed from the source database.
- **bed_number:** The specific identification number or code of the bed.
- **bed_status:** The current availability status of the bed.
- **ward_key (FK):** Foreign key linking the bed to its specific ward in Dim_Ward (Snowflake hierarchy).

5. Dim_Employee

- **employee_key (PK):** Surrogate key for the hospital employee/staff.
- **employee_id (NK):** Natural key identifying the employee.
- **employee_name:** Full name of the employee.
- **gender:** Employee's gender.
- **role:** Designated job role or title (e.g., Doctor, Nurse).
- **employment_type:** Type of employment (e.g., Full-time, Contract).
- **date_of_joining:** The date the employee joined the hospital.
- **department_key (FK):** Foreign key linking the employee to their assigned department in Dim_Department.

6. Dim_Disease

- **disease_key (PK):** Surrogate key for clinical diagnosis.
- **disease_id (NK):** Source system disease identifier.

- **disease_name:** Clinical name of the illness (e.g., Acute Myocardial Infarction).
- **disease_category:** Broader medical category (e.g., Cardiac, Respiratory).

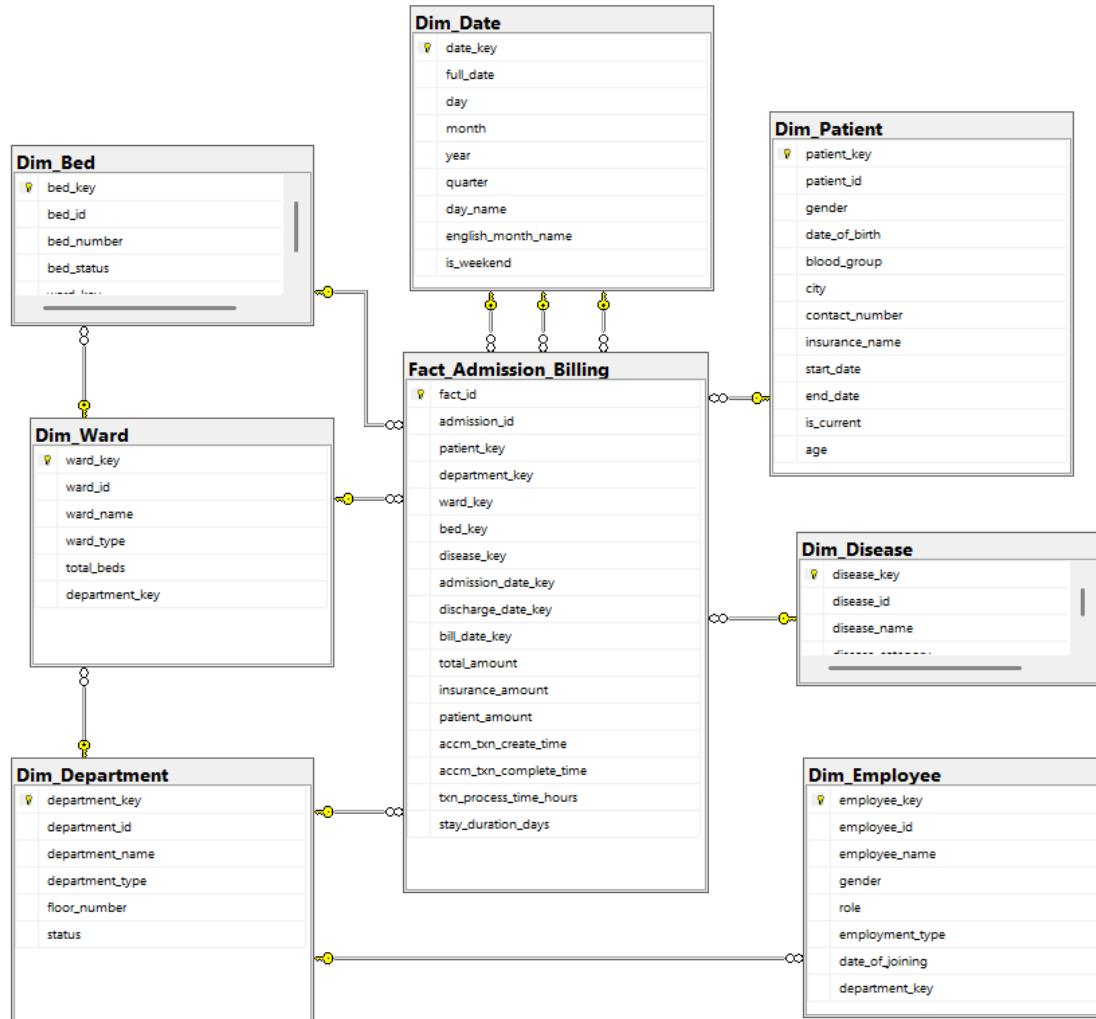
7. Dim_Drug

- **drug_key (PK):** Surrogate key for the pharmaceutical drug.
- **drug_id (NK):** Natural key derived from the source pharmacy system.
- **drug_name:** Generic or scientific name of the drug.
- **brand_name:** Commercial brand name of the drug.
- **drug_category:** Medical classification of the drug.
- **unit_cost:** Standard cost per unit of the drug.

8. Dim_Date

- **date_key (PK):** Unique identifier for each day in YYYYMMDD integer format.
- **full_date:** The complete standard date value (Date format).
- **year / quarter / month / day:** Standard time-based attributes for slicing and drilling down data.
- **day_name / english_month_name:** Descriptive time attributes for intuitive reporting (e.g., Monday, August).
- **is_weekend:** Boolean flag to distinguish between weekdays and weekends for operational analysis.

4.4 Snowflakes Schema Relational Diagram



4.5 Design Assumptions

1. **Insurance Aggregation:** It is assumed that if a patient is covered by multiple providers, they should be viewed as a comma-separated list within the patient dimension for descriptive reporting.
2. **Surrogate Keys:** All dimensions use integer-based surrogate keys (e.g., PatientKey) instead of natural keys to ensure the warehouse is independent of source system changes.
3. **Grain Definition:** The grain of the Fact_Admission_Billing table is defined as **"One row per admission event."**
4. **Date Range:** The Dim_Date table is assumed to cover all historical admission dates and future discharge dates for the next 10 years.
- 5.

Task 5: ETL Development

This ETL (Extract, Transform, Load) process is designed to consolidate data from multiple disparate sources, apply necessary business logic and data cleansing, and finally load the structured data into the Hospital_DW_DB (Data Warehouse).

5.1 Extraction Strategy & Source Diversity

To simulate a sophisticated enterprise environment, the architecture integrates three distinct types of data sources to capture both transactional and master data:

- Relational Database (SQL Server): Core patient records and admission logs are extracted from the Hospital_Staging_DB database using OLE DB Source components.
- Flat Files (CSV): External data, such as patient_insurance.csv and insurance_provider.csv, are ingested to map patients to their respective insurance coverage.
- Text Files (TXT): Detailed billing summaries are extracted from billing_detail.txt to provide granular financial insights.

Extracted data from the Sources.



Load_Dimensio....dtsx [Design]

ParametersEvent HandlersPackage

Execute SQL Task

Extract Department to Staging

Extract Ward to Staging

Extract Bed to Staging

Extract Disease to Staging

Extract Employee to Staging

Extract Doctor to Staging

Extract Drug to Staging

Extract Admission to Staging

Execute SQL Task Editor

Configure the properties required to run SQL statements and stored procedures using the selected connection.

GeneralParameter MappingResult SetExpressions

General

Name	Execute SQL Task
Description	Execute SQL Task

Options

Enter SQL Query

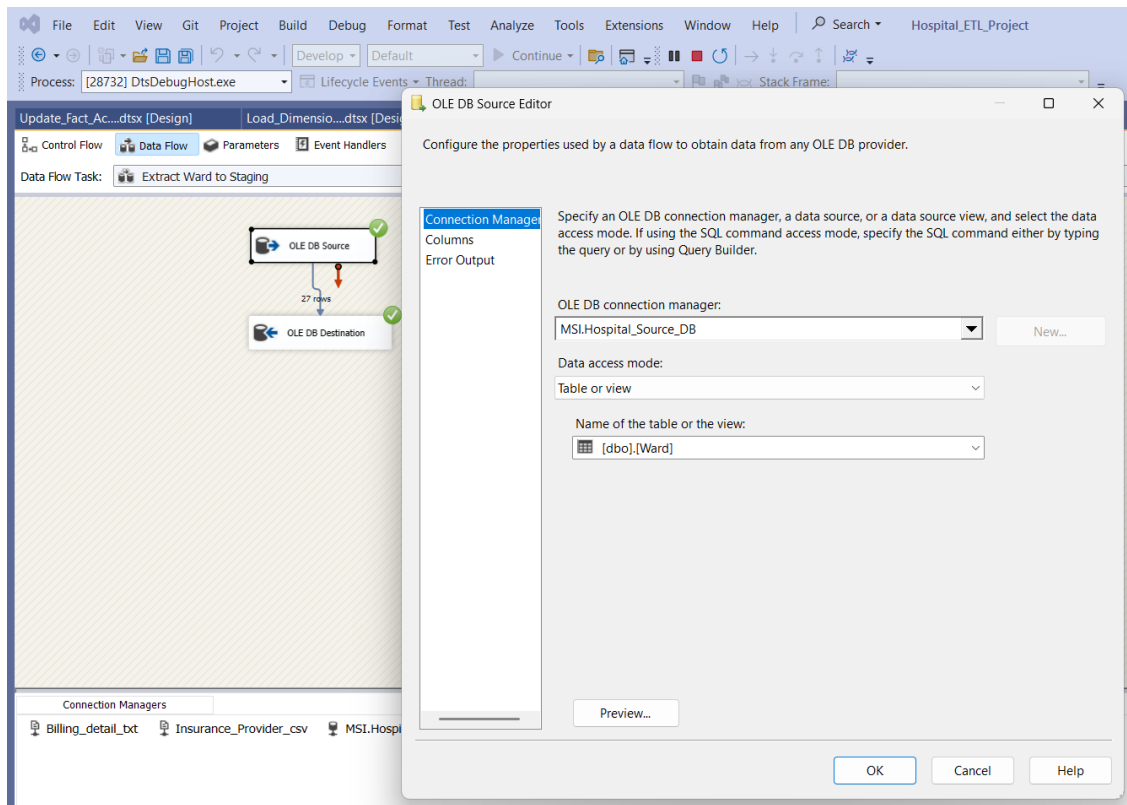
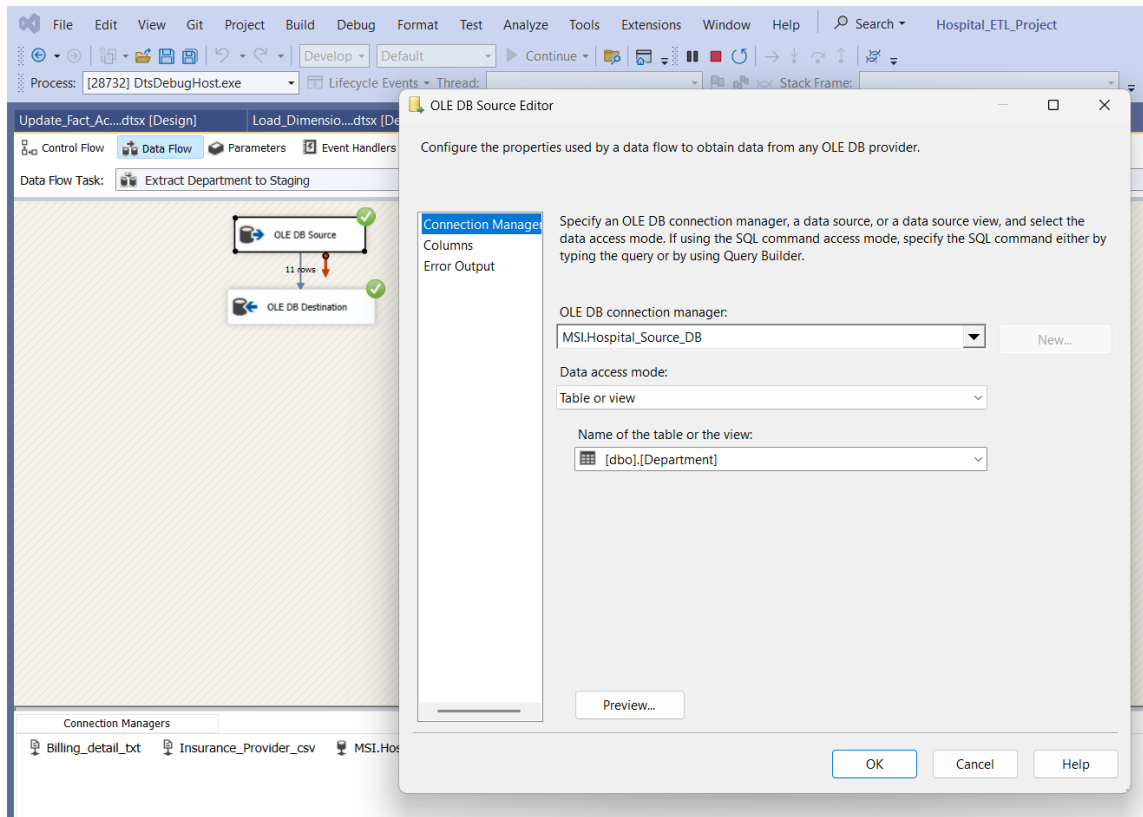
-- 1. Master Data Tables
IF OBJECT_ID('stg_Department', 'U') IS NOT NULL TRUNCATE TABLE stg_Department;
IF OBJECT_ID('stg_Ward', 'U') IS NOT NULL TRUNCATE TABLE stg_Ward;
IF OBJECT_ID('stg_Bed', 'U') IS NOT NULL TRUNCATE TABLE stg_Bed;
IF OBJECT_ID('stg_Disease', 'U') IS NOT NULL TRUNCATE TABLE stg_Disease;
IF OBJECT_ID('stg_Employee', 'U') IS NOT NULL TRUNCATE TABLE stg_Employee;
IF OBJECT_ID('stg_Doctor', 'U') IS NOT NULL TRUNCATE TABLE stg_Doctor;
IF OBJECT_ID('stg_Drug', 'U') IS NOT NULL TRUNCATE TABLE stg_Drug;

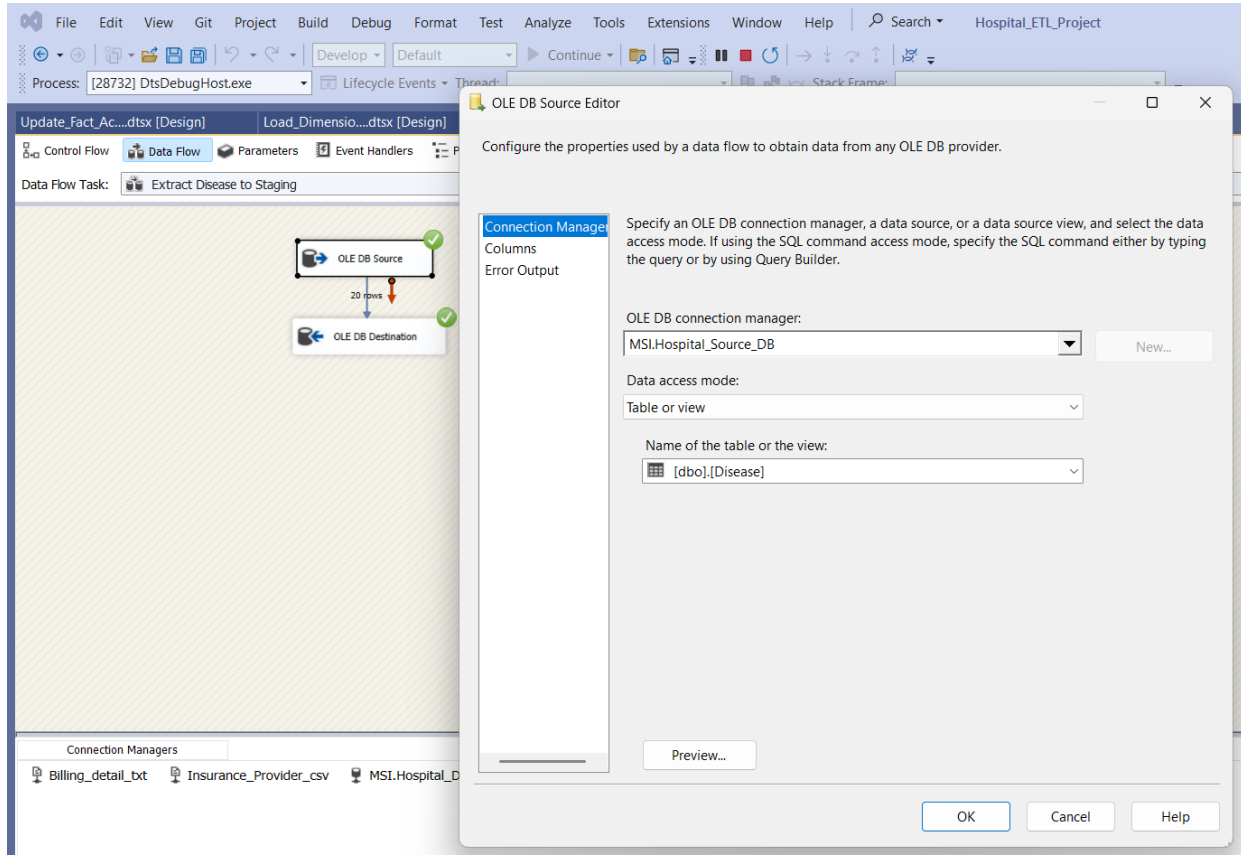
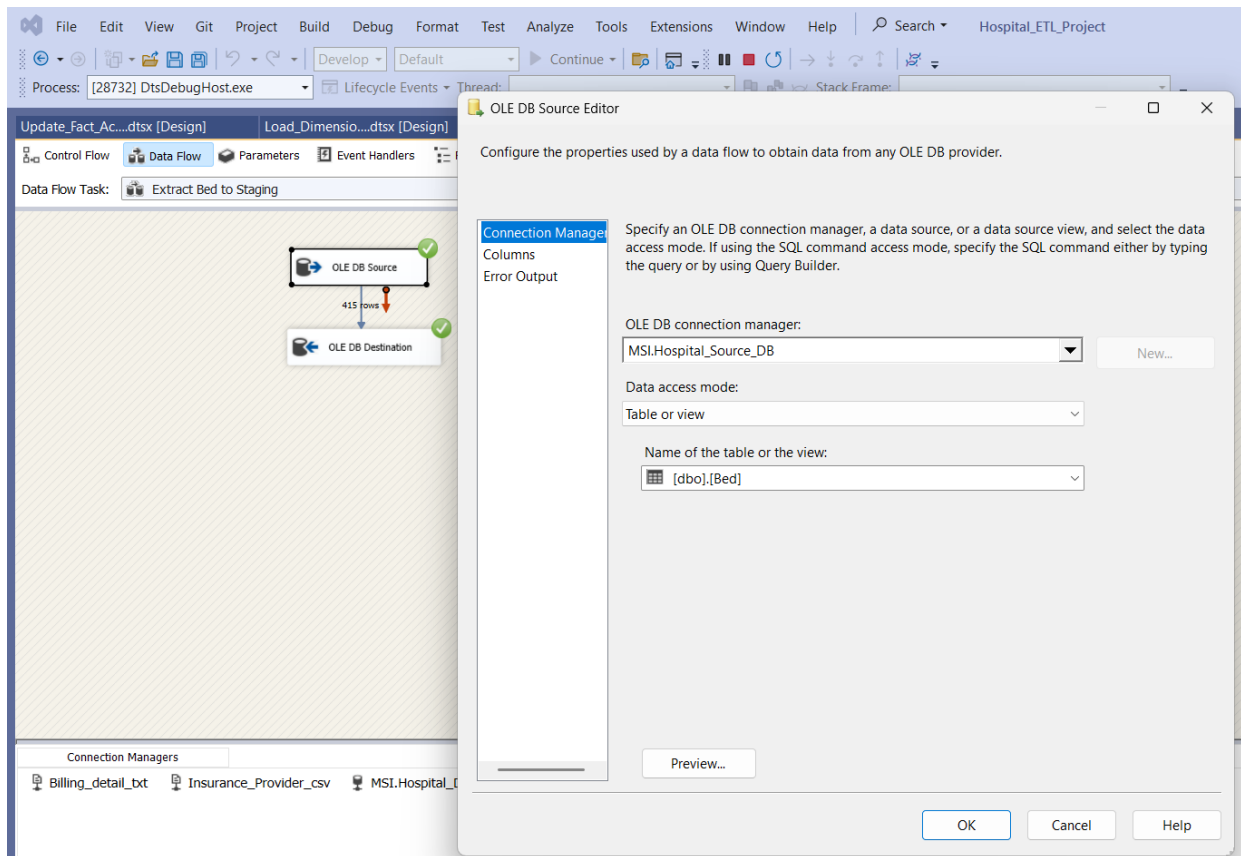
-- 2. Transactional Data Tables
IF OBJECT_ID('stg_Admission', 'U') IS NOT NULL TRUNCATE TABLE stg_Admission;
IF OBJECT_ID('stg_Billing', 'U') IS NOT NULL TRUNCATE TABLE stg_Billing;
IF OBJECT_ID('stg_Prescription', 'U') IS NOT NULL TRUNCATE TABLE stg_Prescription;
IF OBJECT_ID('stg_Staff_assignment', 'U') IS NOT NULL TRUNCATE TABLE stg_Staff_assignment;
IF OBJECT_ID('stg_Drug_inventory', 'U') IS NOT NULL TRUNCATE TABLE stg_Drug_inventory;

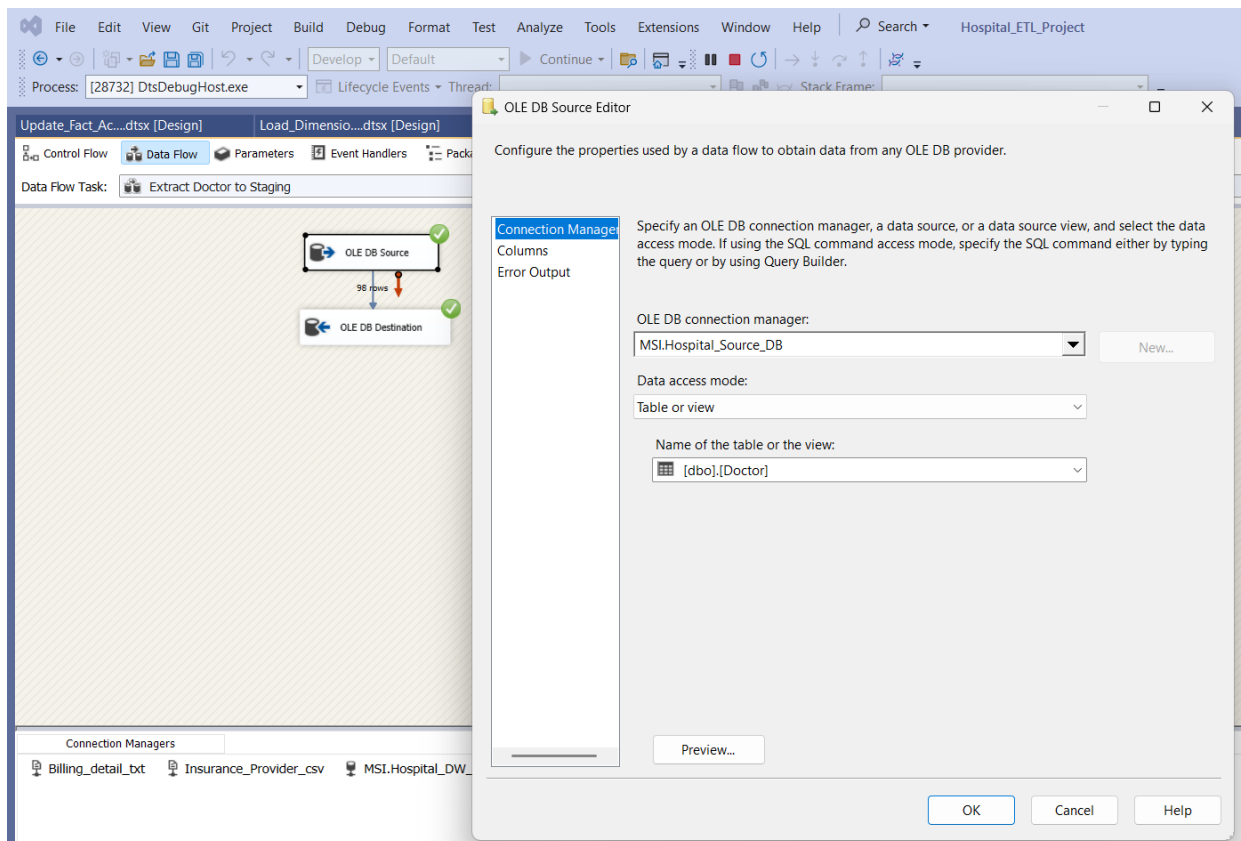
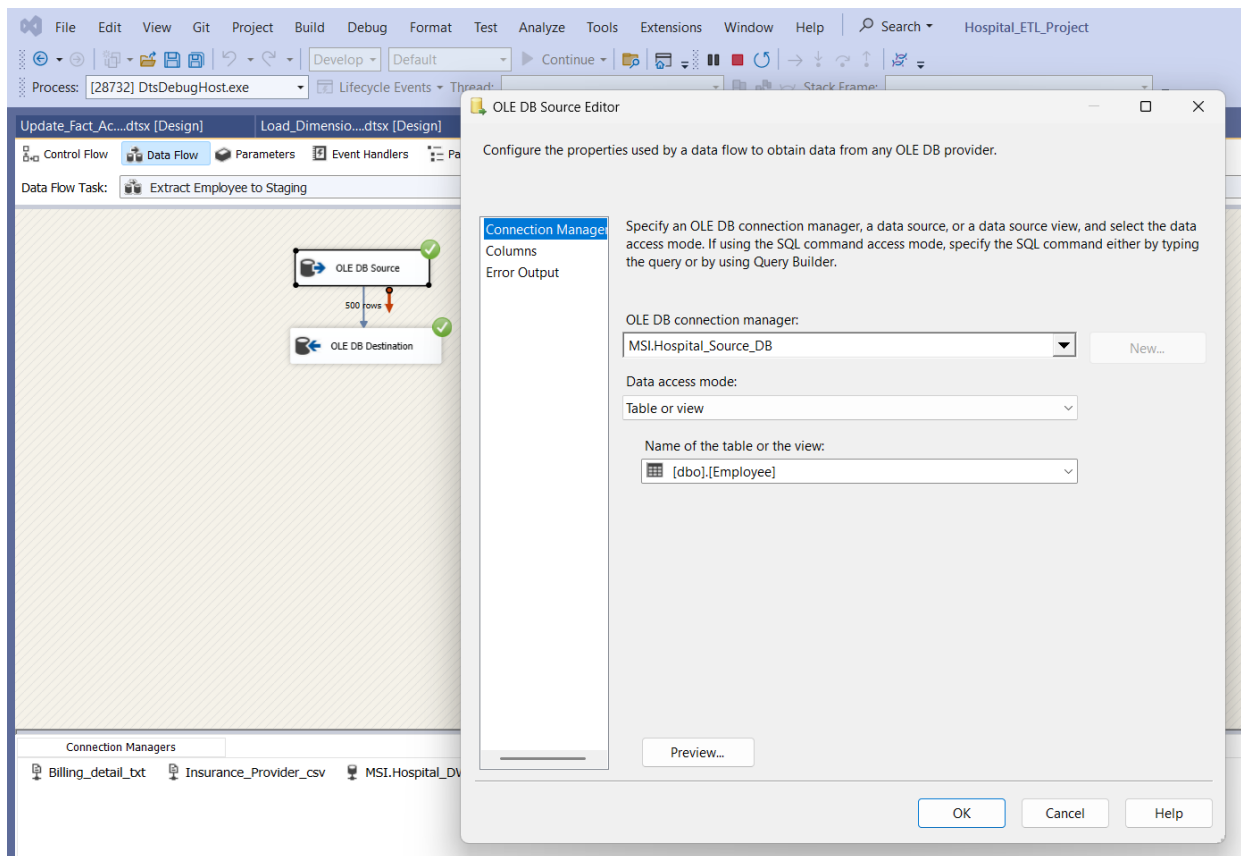
-- 3. Flat File Source Tables
IF OBJECT_ID('stg_Patient', 'U') IS NOT NULL TRUNCATE TABLE stg_Patient;
IF OBJECT_ID('stg_Insurance_provider', 'U') IS NOT NULL TRUNCATE TABLE stg_Insurance_provider;
IF OBJECT_ID('stg_Patient_insurance', 'U') IS NOT NULL TRUNCATE TABLE stg_Patient_insurance;
IF OBJECT_ID('stg_Billing_detail', 'U') IS NOT NULL TRUNCATE TABLE stg_Billing_detail];

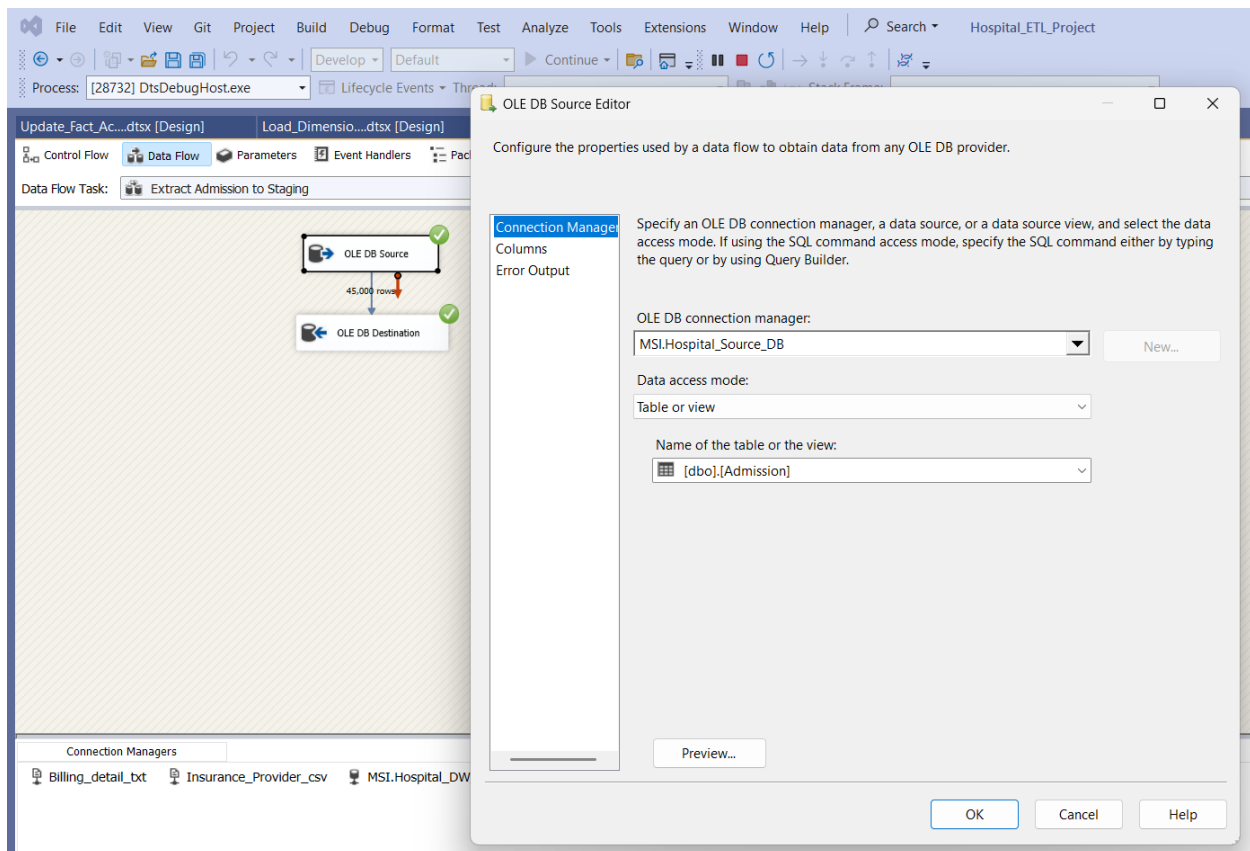
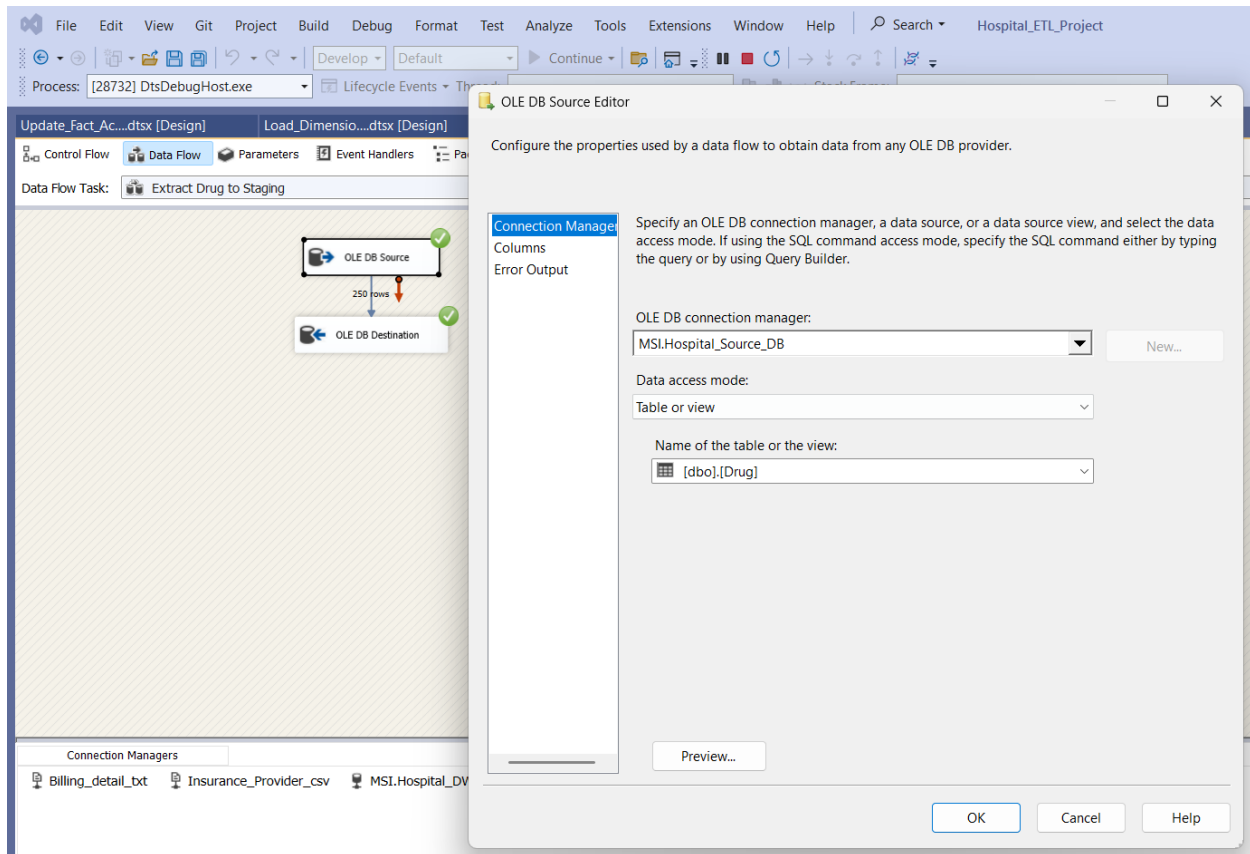
OKCancel

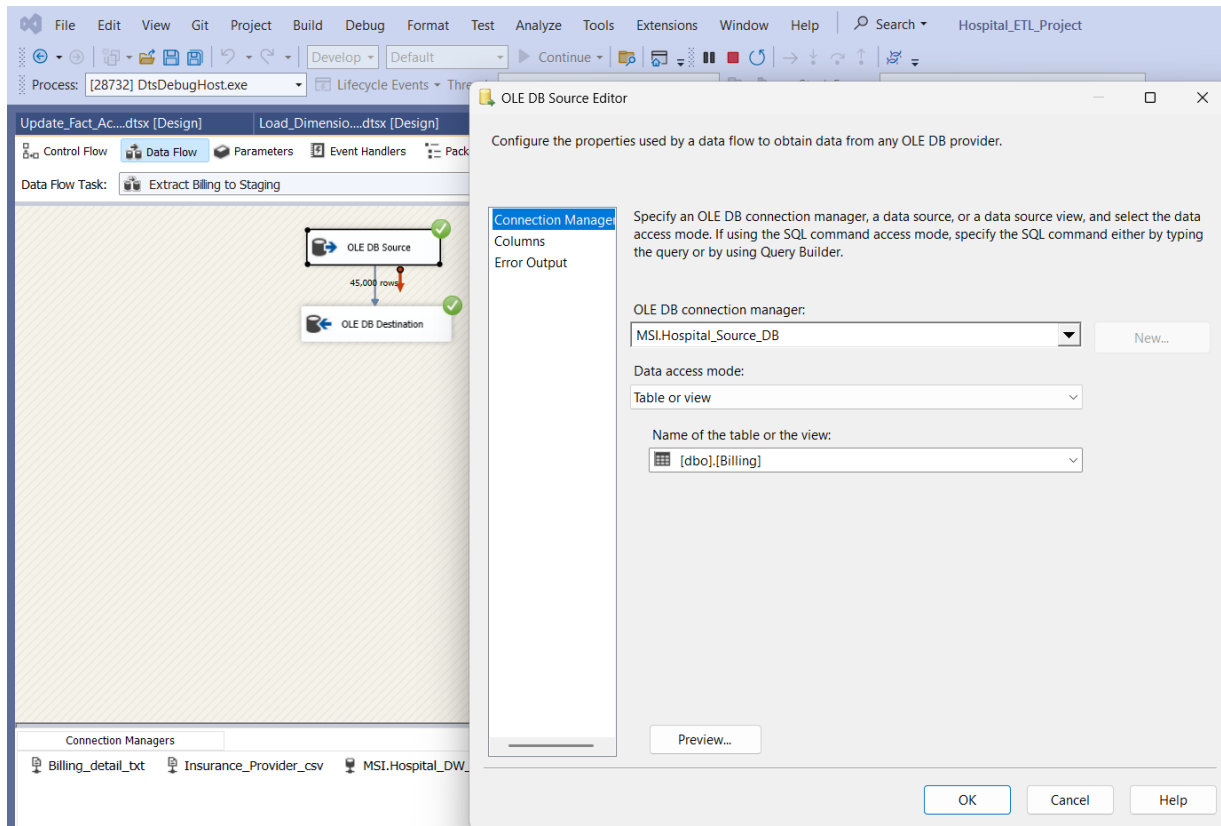
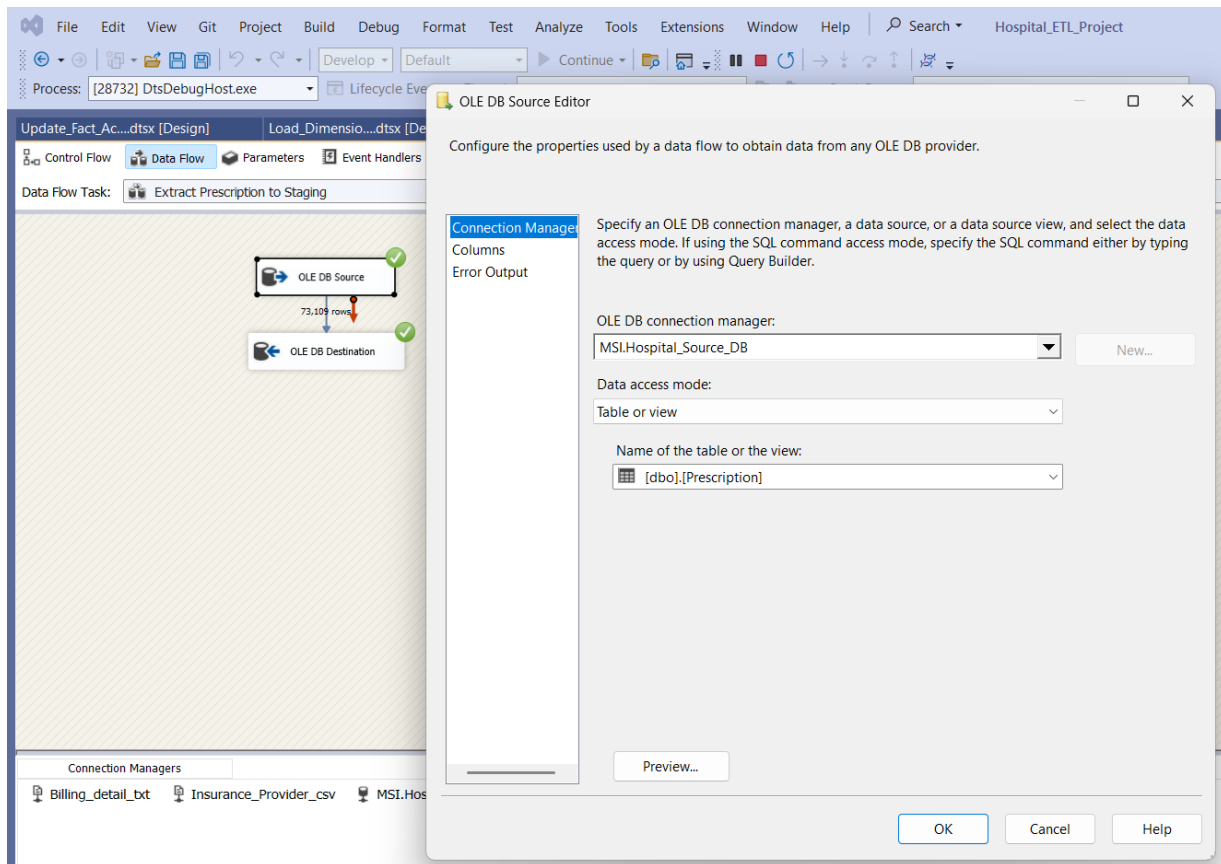
- From Hospital_staging_DB

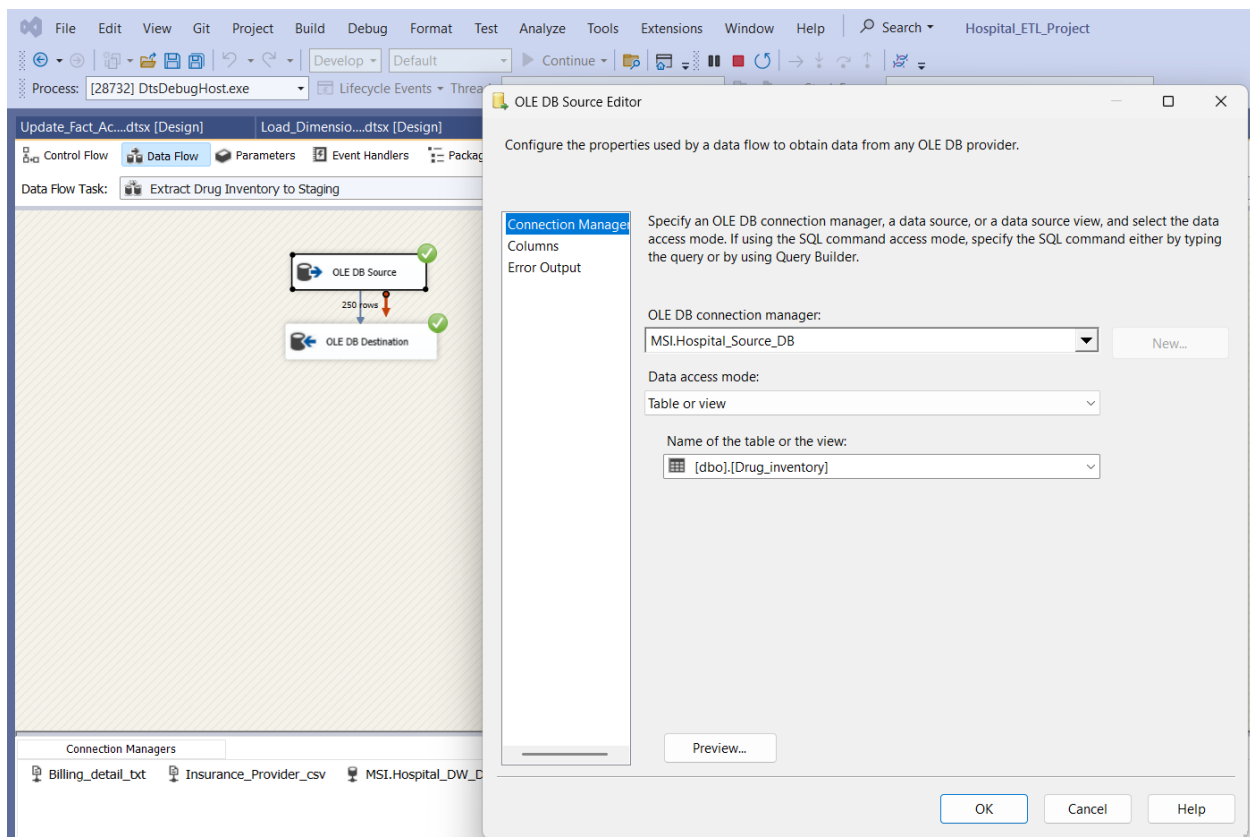
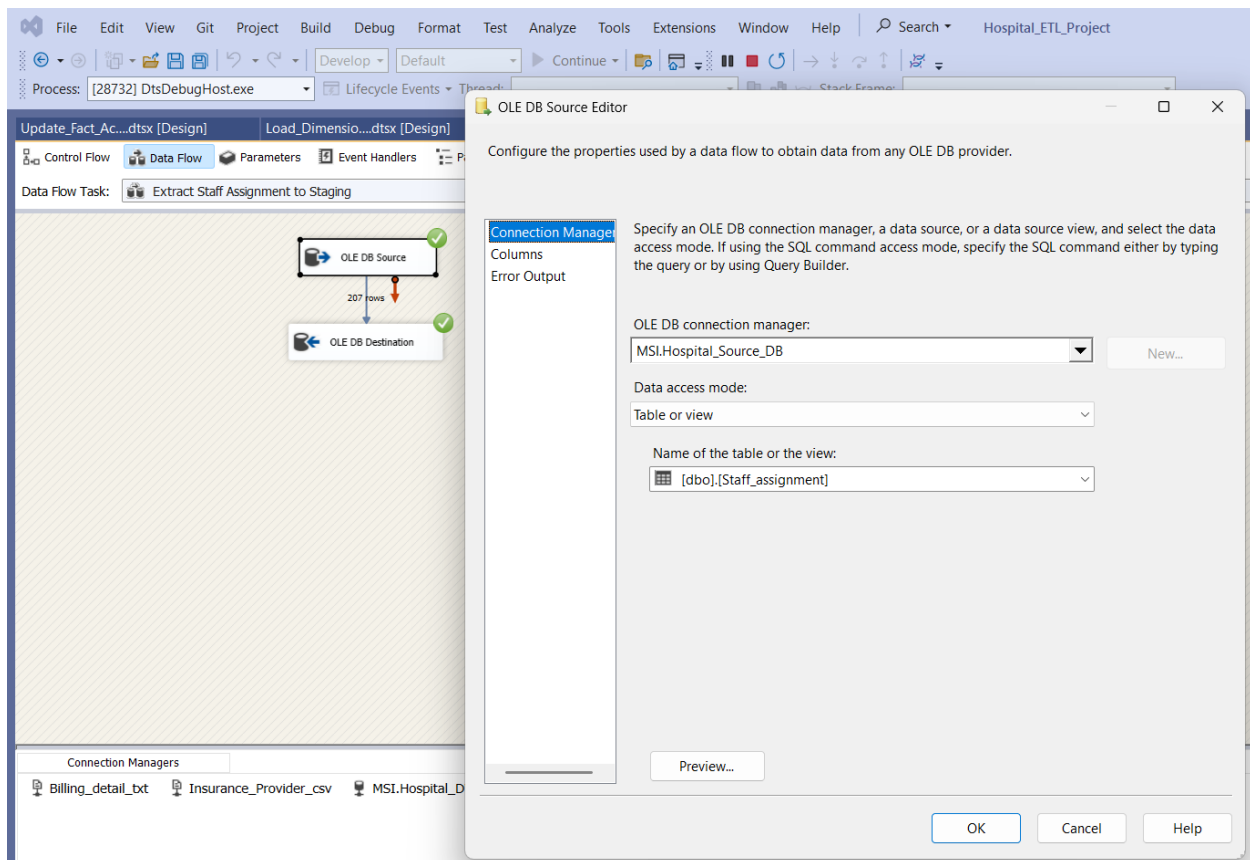




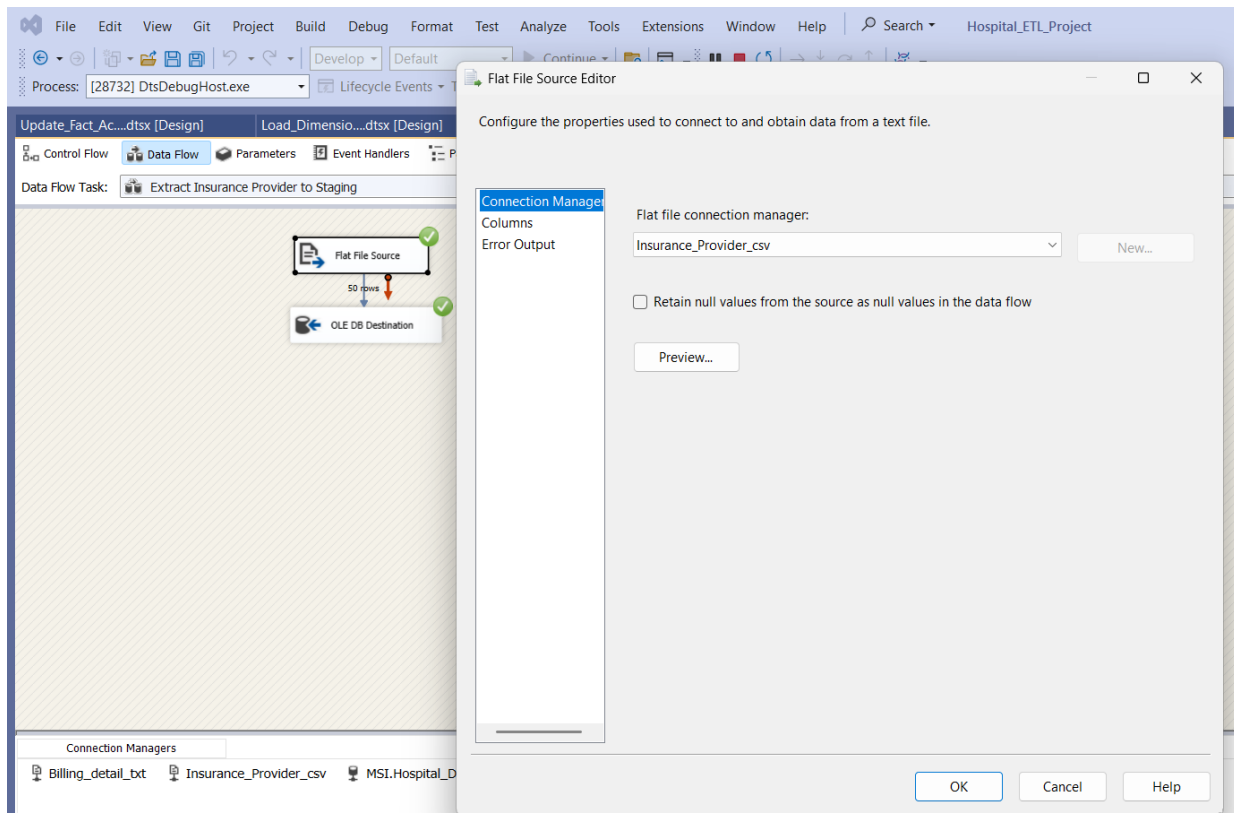
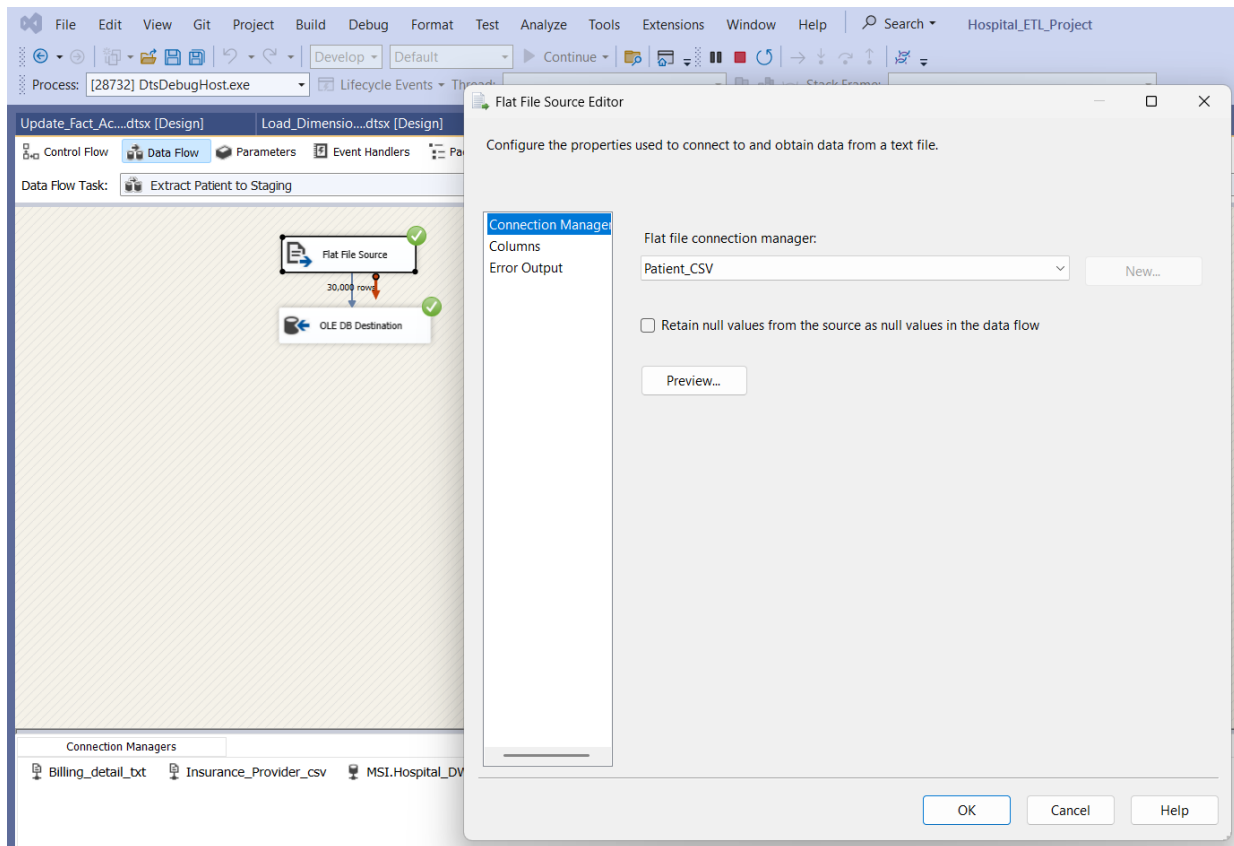


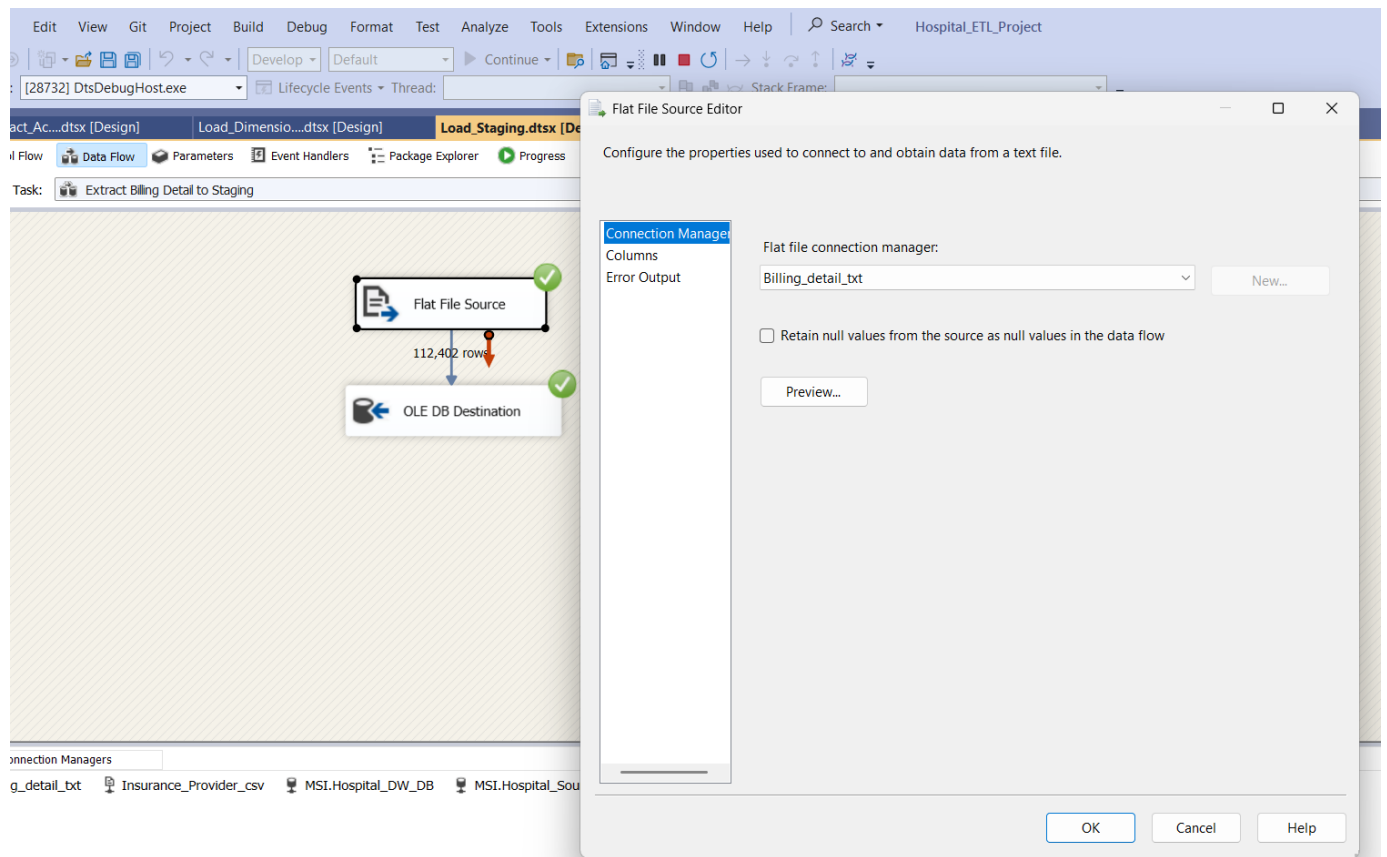
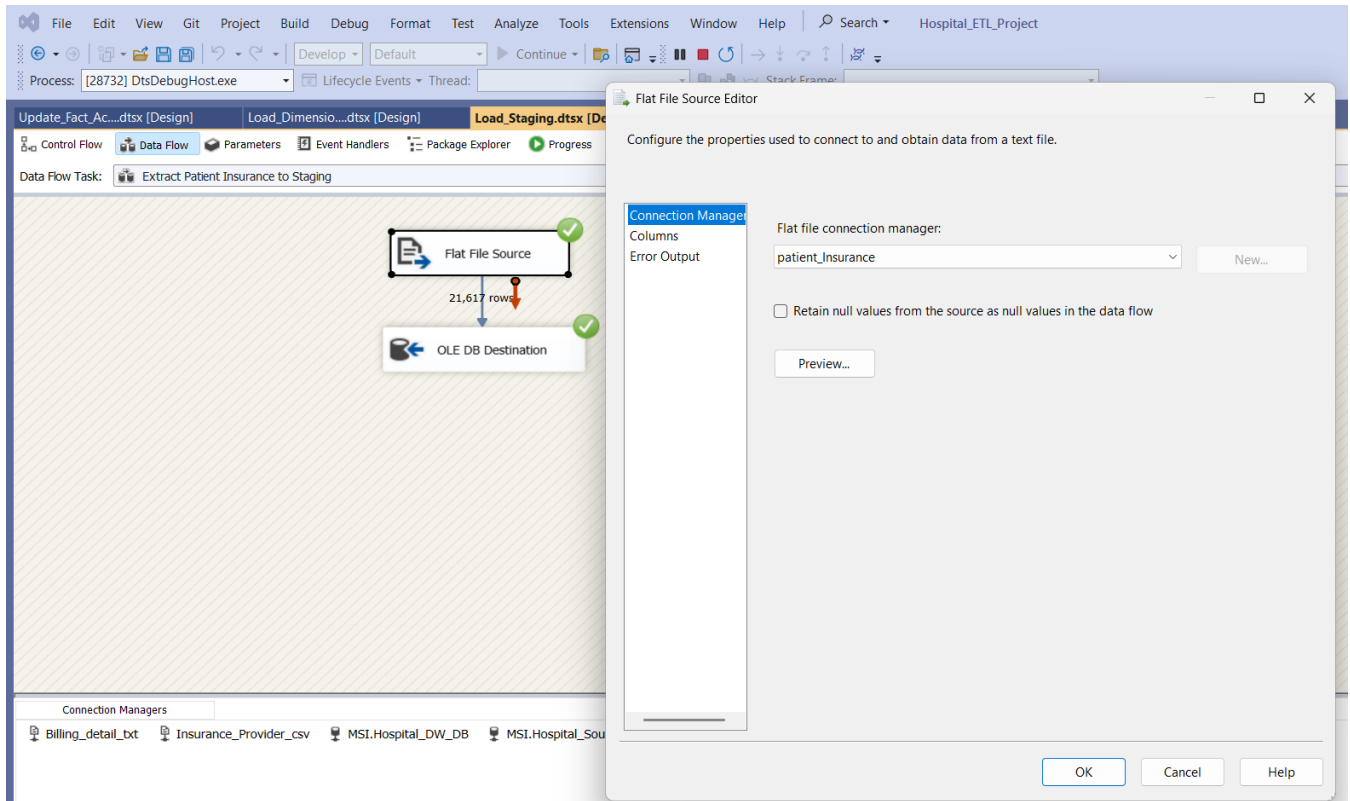






- From the Flat Files





The screenshot shows a SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure, including tables like `dbo.stg_Admission`, `dbo.stg_Bed`, `dbo.stg_Billing`, `dbo.stg_Billing_detail`, `dbo.stg_Department`, `dbo.stg_Disease`, `dbo.stg_Doctor`, `dbo.stg_Drug`, `dbo.stg_Drug_inventory`, `dbo.stg_Employee`, `dbo.stg_Insurance_provider`, `dbo.stg_Patient`, `dbo.stg_Patient_insurance`, `dbo.stg_Prescription`, `dbo.stg_Staff_assignment`, `dbo.stg_Transaction_Updates`, and `dbo.stg_Ward`. The main window shows a SQL query script with the following content:

```
-- Display TOP 10 records from each staging table
SELECT TOP 10 * FROM dbo.stg_Admission;
SELECT TOP 10 * FROM dbo.stg_Bed;
SELECT TOP 10 * FROM dbo.stg_Billing;
SELECT TOP 10 * FROM dbo.stg_Billing_detail;
SELECT TOP 10 * FROM dbo.stg_Department;
SELECT TOP 10 * FROM dbo.stg_Disease;
SELECT TOP 10 * FROM dbo.stg_Doctor;
SELECT TOP 10 * FROM dbo.stg_Drug;
SELECT TOP 10 * FROM dbo.stg_Drug_inventory;
SELECT TOP 10 * FROM dbo.stg_Employee;
SELECT TOP 10 * FROM dbo.stg_Insurance_provider;
SELECT TOP 10 * FROM dbo.stg_Patient;
SELECT TOP 10 * FROM dbo.stg_Patient_insurance;
SELECT TOP 10 * FROM dbo.stg_Prescription;
SELECT TOP 10 * FROM dbo.stg_Staff_assignment;
SELECT TOP 10 * FROM dbo.stg_Transaction_Updates;
SELECT TOP 10 * FROM dbo.stg_Ward;
```

The query is executed successfully, and the results are displayed in a grid. The first table shows admission records with columns: `admission_id`, `admission_date`, `discharge_date`, `admission_type`, `admission_status`, `patient_id`, `department_id`, `ward_id`, `bed_id`, and `disease_id`. The second table shows bed status with columns: `bed_id`, `bed_number`, `bed_status`, and `ward_id`. The third table shows billing records with columns: `bill_id`, `bill_date`, `total_amount`, `insurance_covered_amount`, `patient_payable_amount`, `payment_status`, `payment_mode`, and `admission_id`.

5.2 Order of Execution

Maintaining the correct sequence of execution is critical to ensure Referential Integrity. The loading process follows a strict hierarchy:

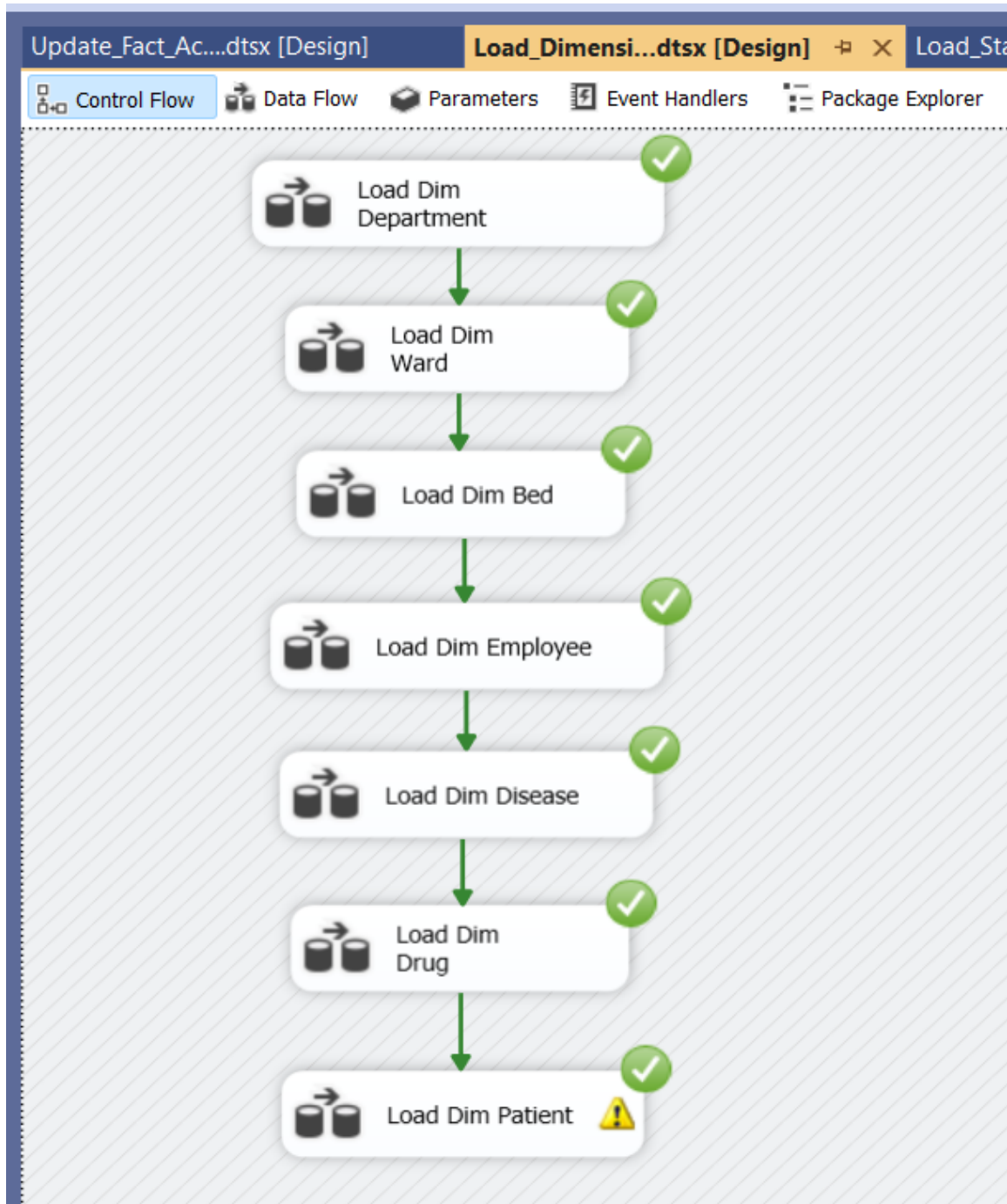
Dimension tables must be populated first because the Fact table relies on them to generate its Surrogate Keys. Furthermore, due to the Snowflake schema architecture, the loading sequence strictly follows referential integrity rules (parent dimensions must be loaded before child dimensions):

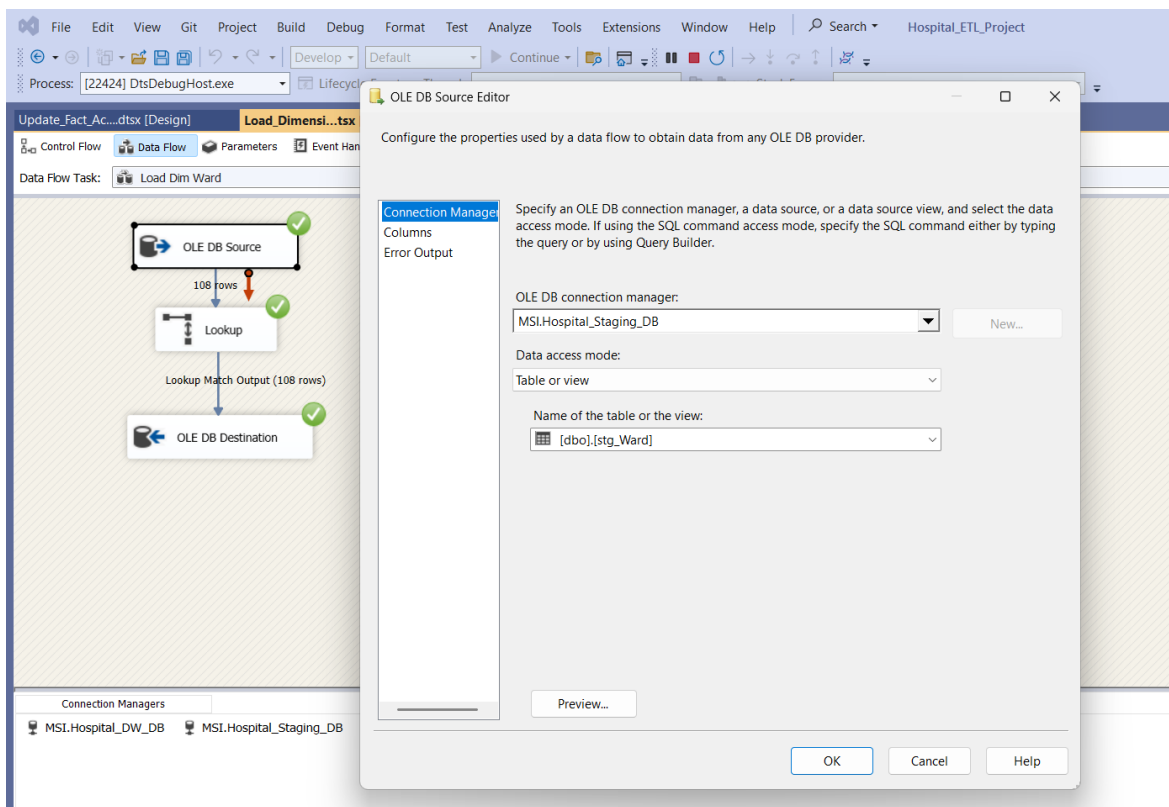
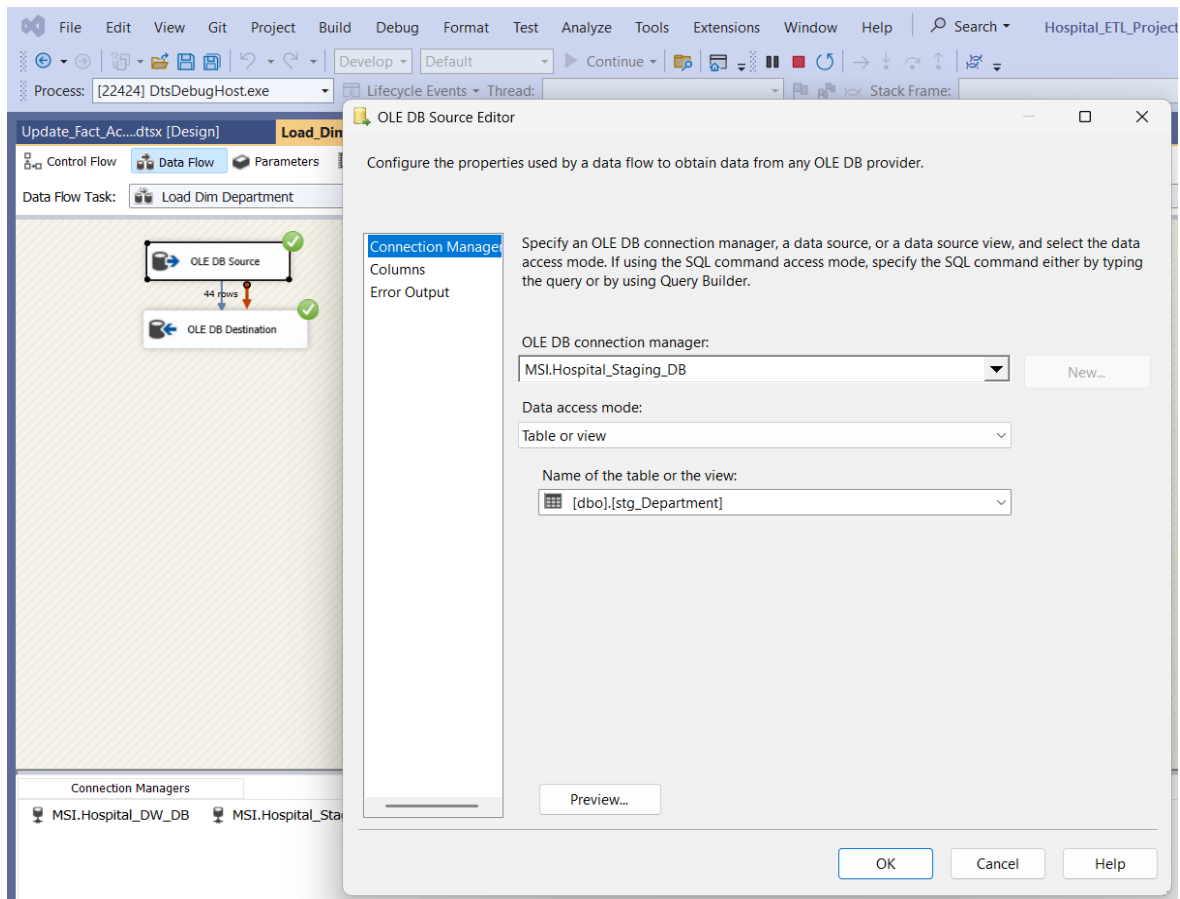
- **Dim_Department, Dim_Ward & Dim_Bed:** Loaded sequentially to maintain hierarchical dependencies. `Dim_Department` is populated first, providing the necessary foreign keys for `Dim_Ward`, which in turn allows `Dim_Bed` to be successfully loaded.
- **Dim_Employee, Dim_Disease & Dim_Drug:** Loaded subsequently to categorize hospital staff, clinical diagnoses, and pharmaceutical details.
- **Dim_Patient:** Implemented last in this sequence using a Slowly Changing Dimension (SCD) transformation. This manages patient history by handling Type 1 updates (overwriting current attributes like age, city, and contact_number) and Type 2 updates (creating new historical records for insurance_name changes).
- **Dim_Date:** (Note: Populated via a standalone execution) Serves as a static, time-based reference for all hospital activities and is required before any Fact table loading begins.

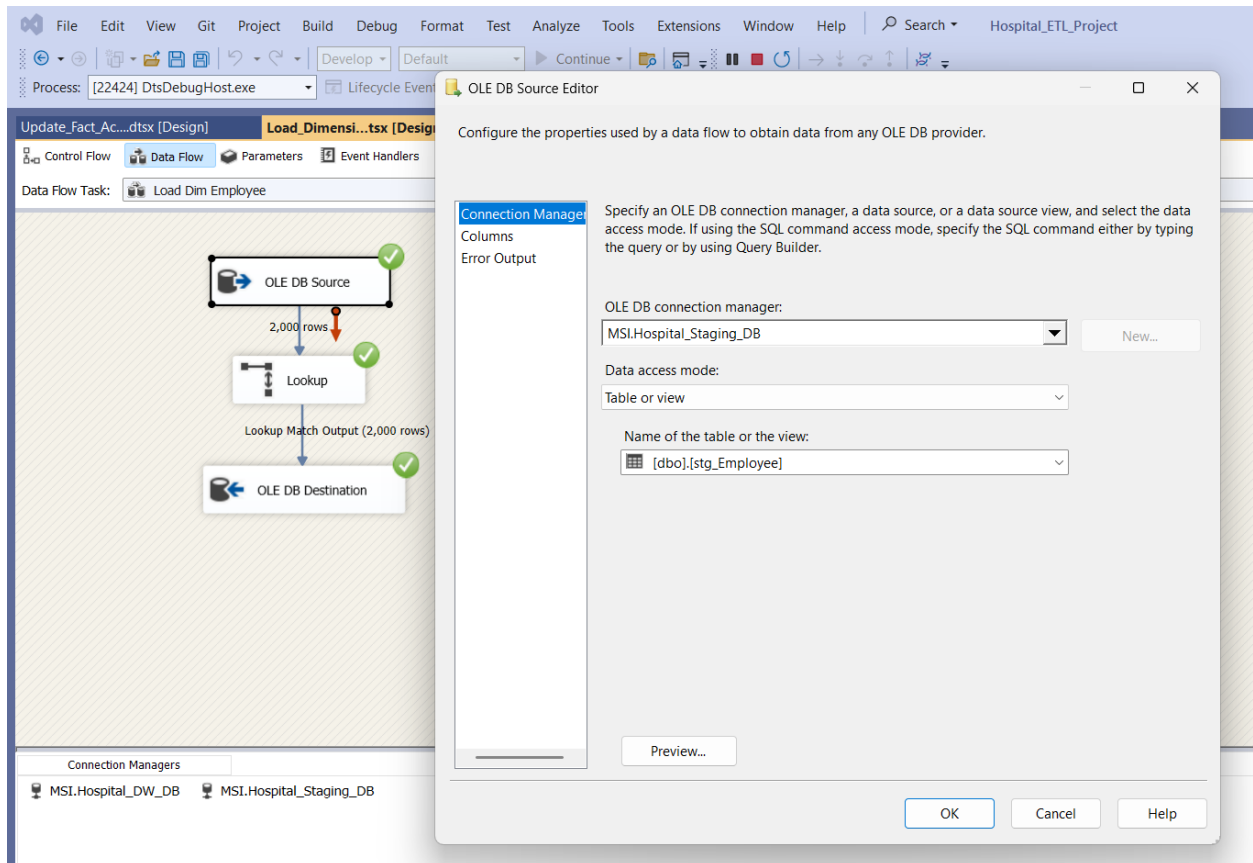
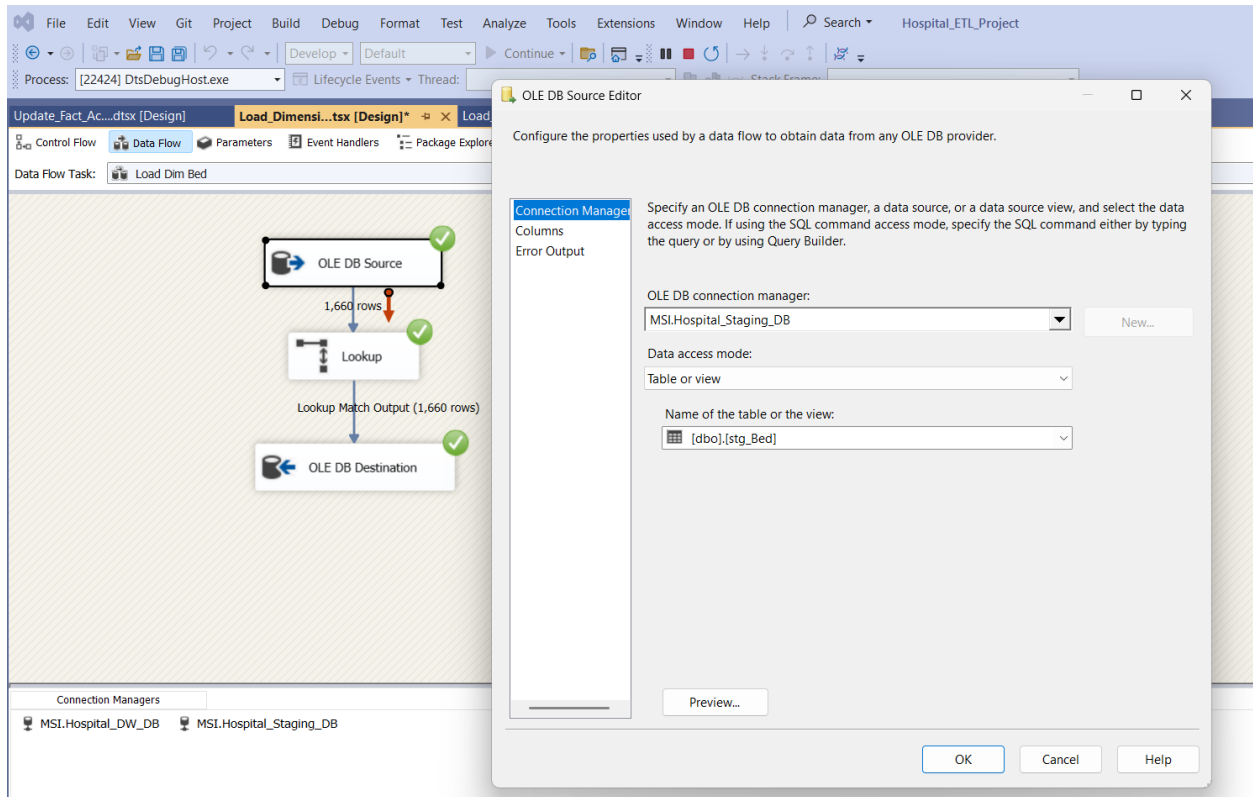
Phase 2: Fact Table Loading (Secondary Priority)

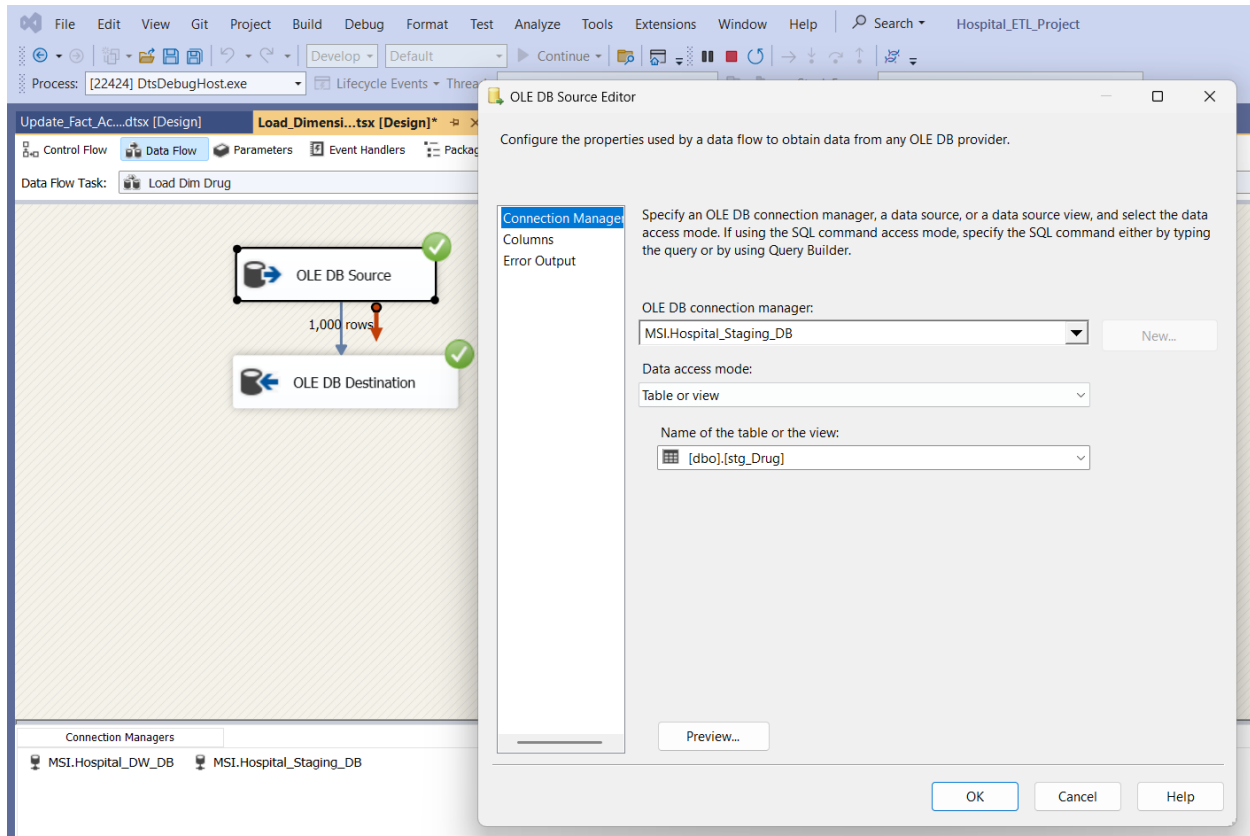
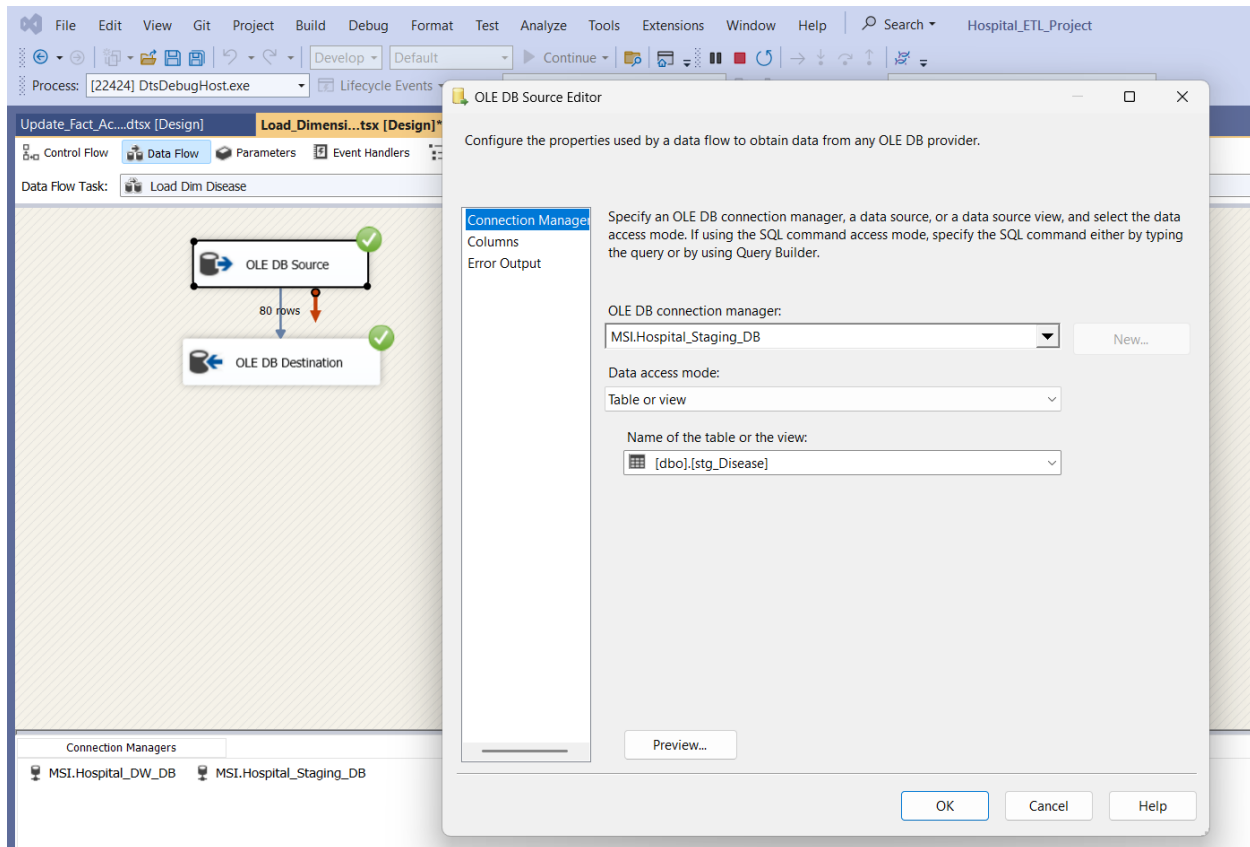
Once all dimensions are verified and loaded, the transactional data is processed.

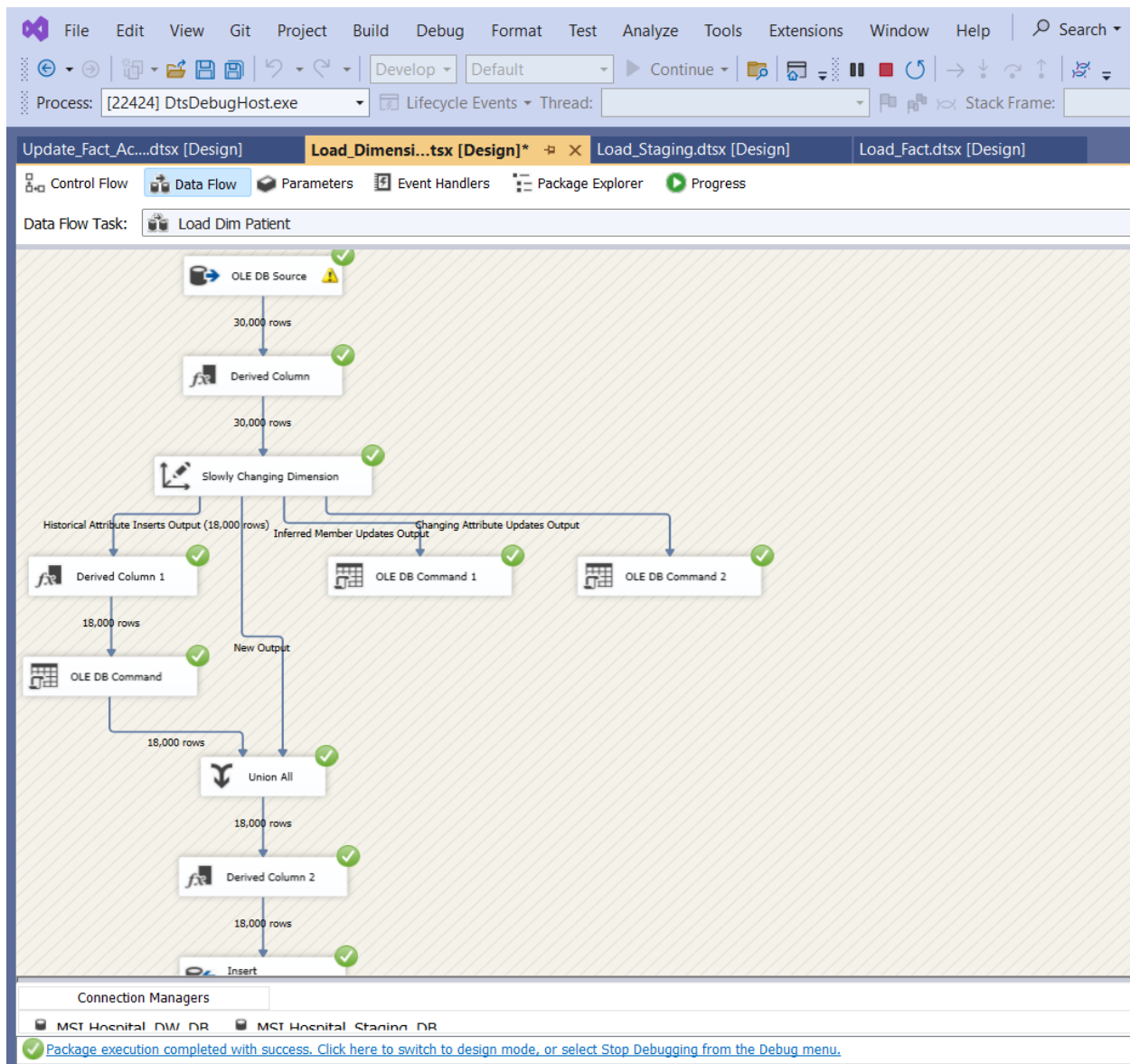
- **Fact_Admission_Billing:** This table aggregates clinical and financial data, linking it to the previously loaded dimensions through surrogate keys.











```

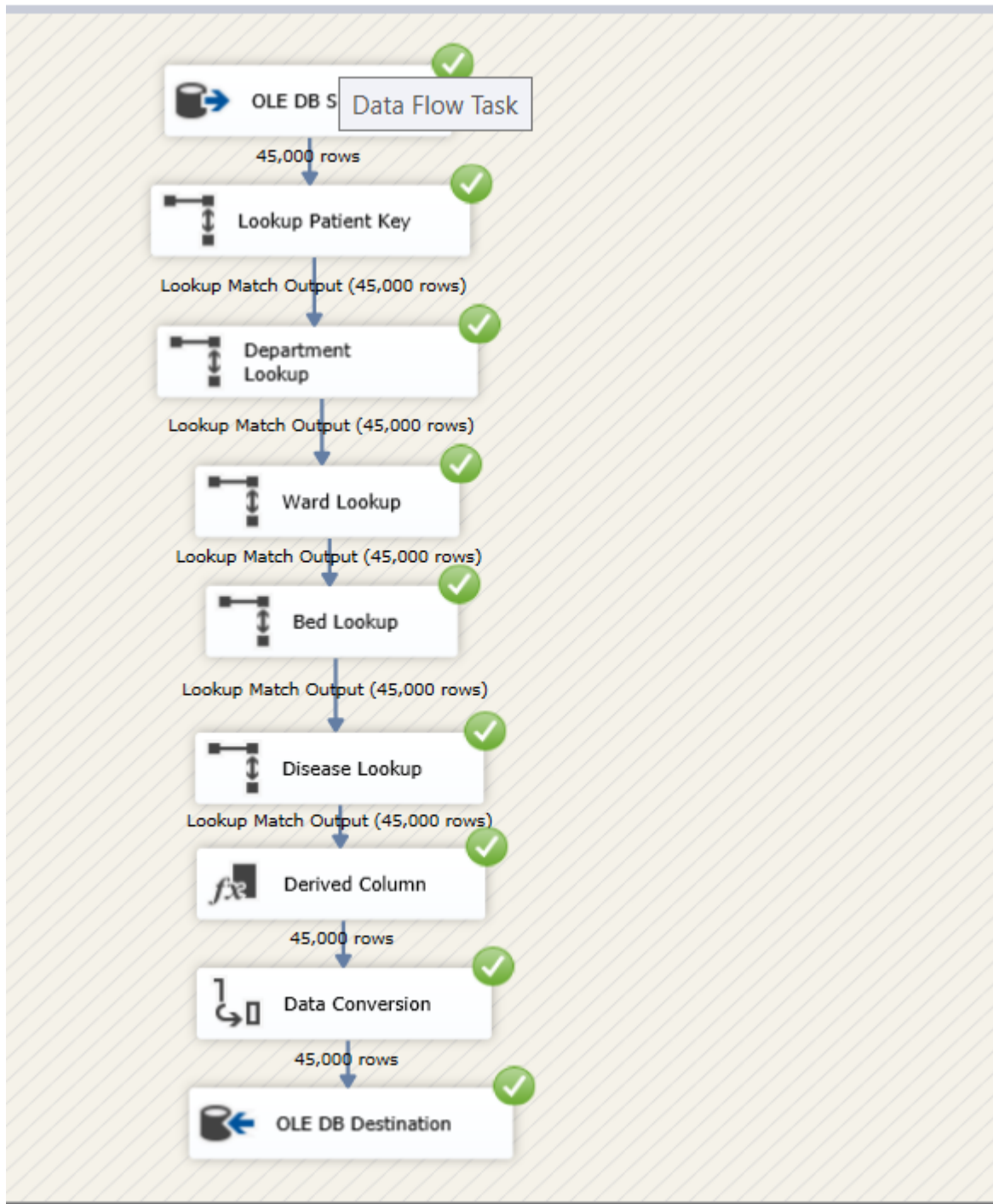
-- create dim date table
CREATE TABLE dbo.Dim_Date (
    date_key INT PRIMARY KEY,          -- Format: YYYYMMDD
    full_date DATE NOT NULL,
    day INT NOT NULL,
    month INT NOT NULL,
    year INT NOT NULL,
    quarter INT NOT NULL,
    day_name VARCHAR(20) NOT NULL,
    english_month_name VARCHAR(20) NOT NULL,
    is_weekend BIT NOT NULL           -- 1 = Weekend, 0 = Weekday
);
GO

-- fill the data to dim date
DECLARE @StartDate DATE = '2020-01-01';
DECLARE @EndDate DATE = '2030-12-31';

WHILE @StartDate <= @EndDate
BEGIN
    INSERT INTO dbo.Dim_Date (
        date_key,
        full_date,
        day,
        month,
        year,
        quarter,
        day_name,
        english_month_name,
        is_weekend
    )
    VALUES (
        CAST(FORMAT(@StartDate, 'yyyyMMdd') AS INT),
        @StartDate,
        DAY(@StartDate),
        MONTH(@StartDate),
        YEAR(@StartDate),
        DATEPART(QUARTER, @StartDate),
        DATENAME(WEEKDAY, @StartDate),
        DATENAME(MONTH, @StartDate),
        CASE WHEN DATEPART(WEEKDAY, @StartDate) IN (1, 7) THEN 1 ELSE 0 END
    );
    SET @StartDate = DATEADD(DAY, 1, @StartDate);
END;
GO

```

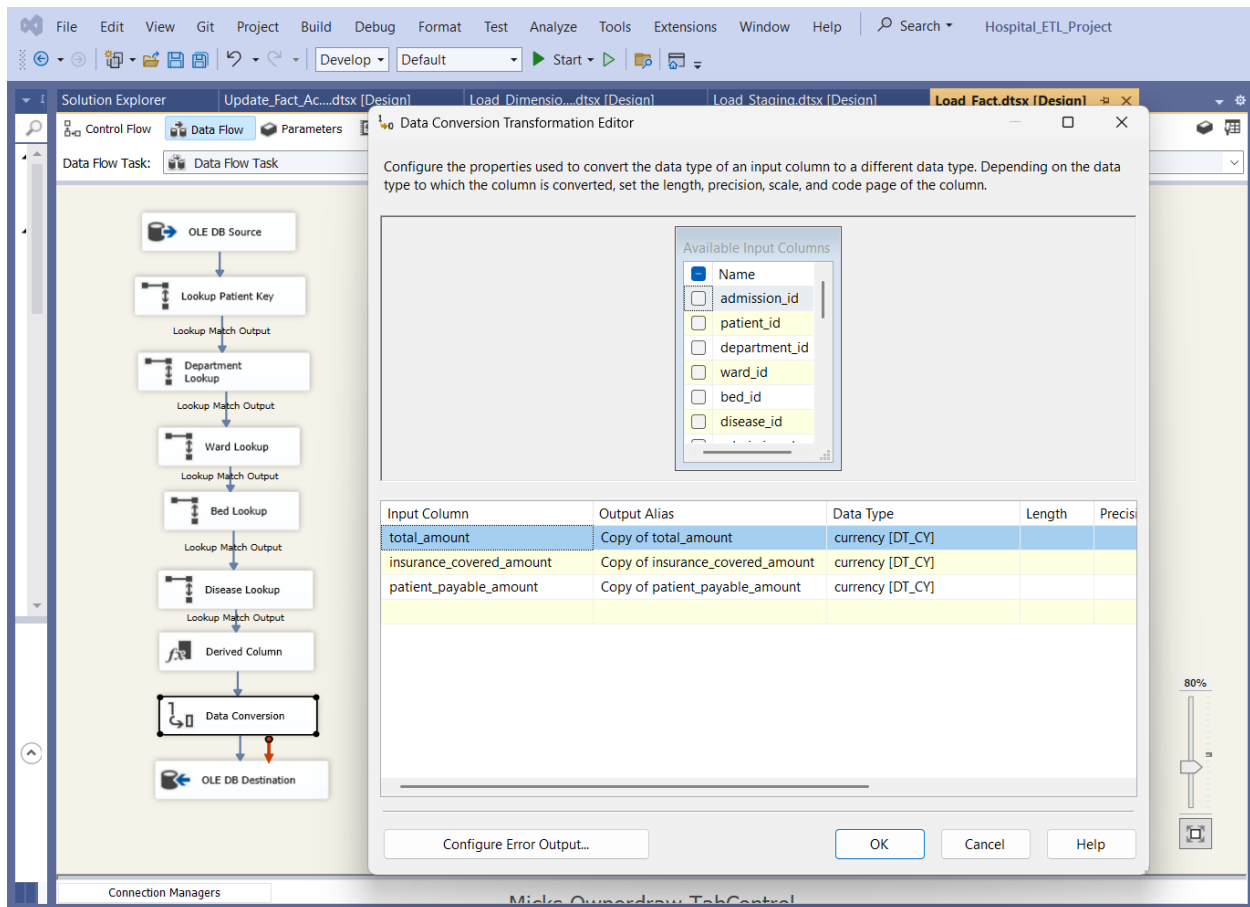
-Fact Table



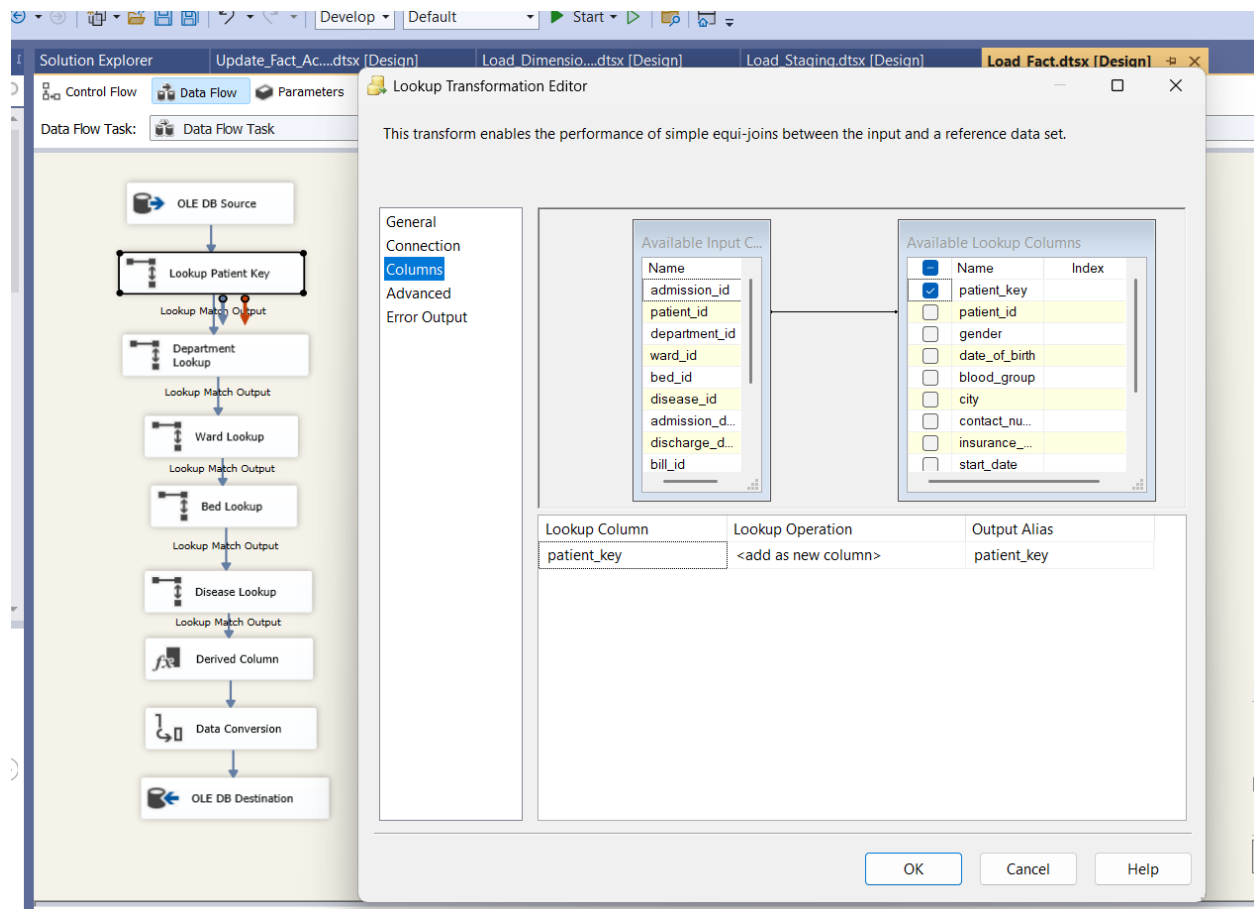
5.3 Key Transformation Logic

The following transformations ensure data quality and business relevance:

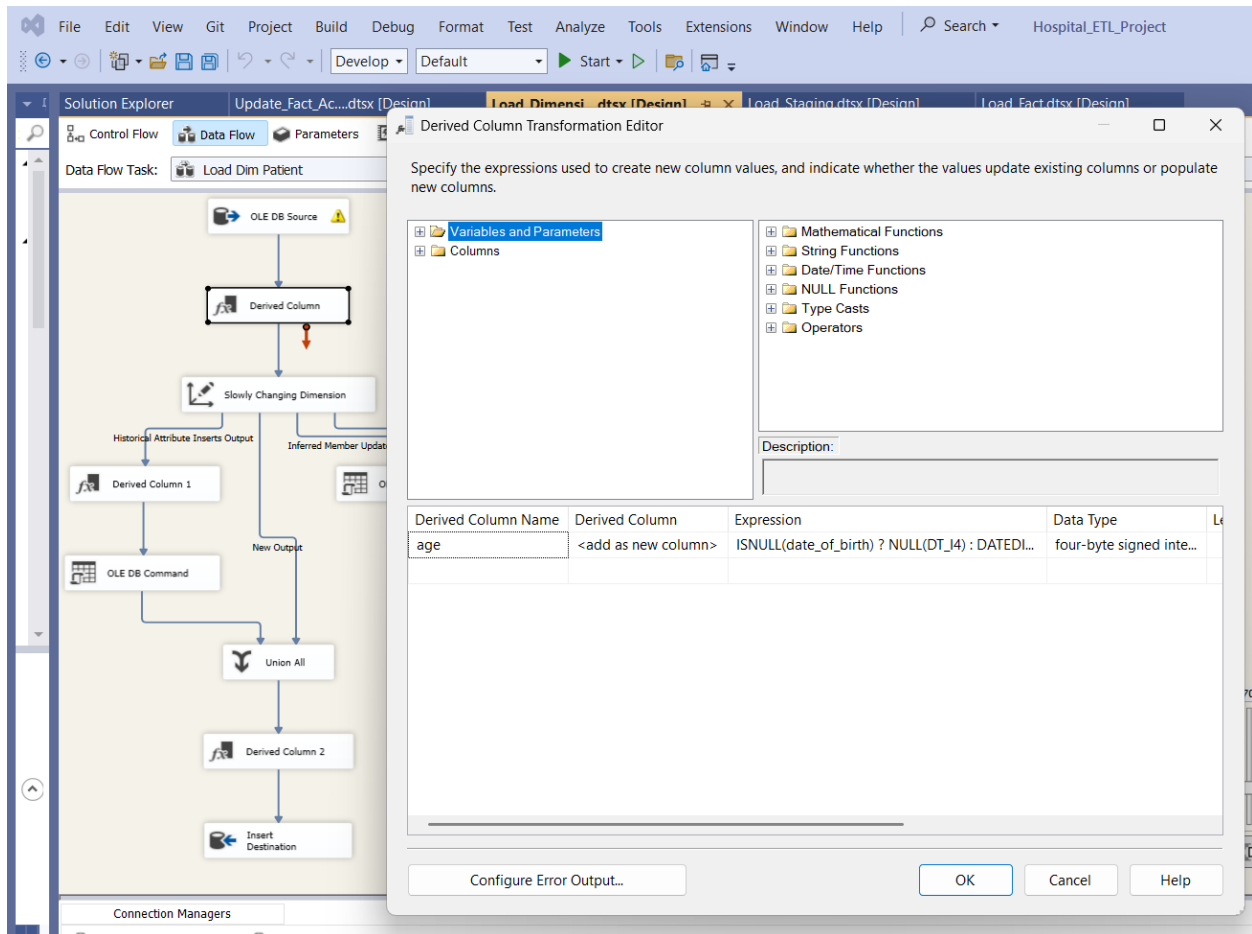
- **Data Conversion:** Used to explicitly cast source data types to match the Fact table's strict schema requirements. In this step, financial measures - specifically `total_amount`, `insurance_covered_amount`, and `patient_payable_amount` - are converted into the Currency data type (`DT_CY`). This ensures mathematical precision and prevents data truncation errors during financial analytics.



- **Lookup Transformations:** These components perform real-time equi-joins between the incoming transactional source data and the pre-loaded Dimension tables to retrieve the correct Surrogate Keys for the Fact table. As demonstrated in the pipeline sequence, individual Lookup transformations are systematically configured to map natural business keys from the source to their respective surrogate keys in the Data Warehouse. Specifically, the pipeline retrieves the patient_key, department_key, ward_key, bed_key, and disease_key by mapping their corresponding source IDs (patient_id, department_id, ward_id, bed_id, and disease_id).



- **Derived Columns:** This transformation is utilized to dynamically calculate new analytical fields, such as computing a patient's age from their date_of_birth. Using an SSIS expression incorporating ISNULL and DATEDIFF, the age is accurately generated as an integer (DT_I4) before being processed by the Slowly Changing Dimension component.



Derived Column 1 (end_date): When a patient's historical attribute changes, this component captures the current system date to mark the expiration date of the old, outdated record.

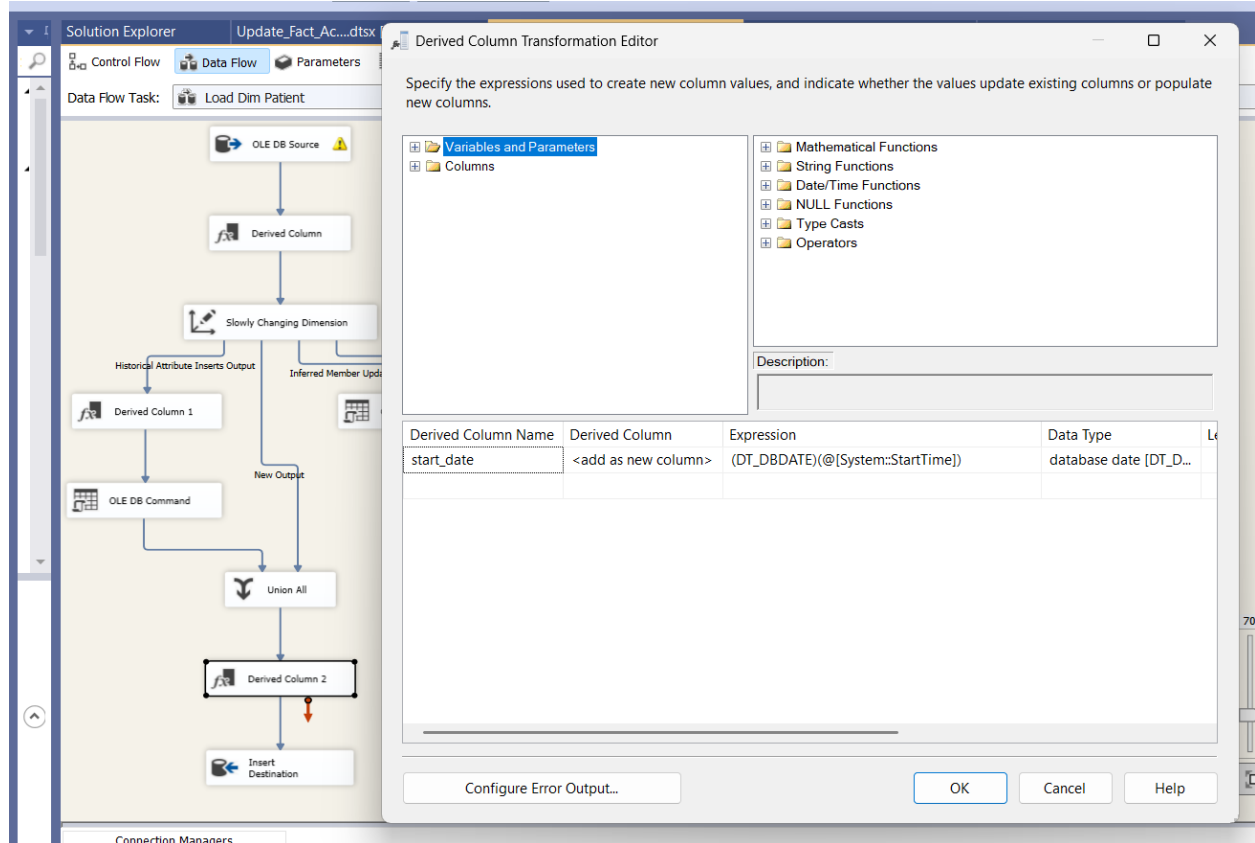
The screenshot displays the SQL Server Data Tools (SSDT) interface for a project named 'Hospital_ETL_Project'. The main window shows a Data Flow Task named 'Load Dim Patient'. The task flow is as follows: OLE DB Source → Derived Column → Slowly Changing Dimension → Derived Column 1 → OLE DB Command → Union All → Derived Column 2 → Insert Destination. The 'Derived Column 1' component is highlighted, and the 'Derived Column Transformation Editor' dialog is open.

The 'Derived Column Transformation Editor' dialog has a title bar 'Derived Column Transformation Editor'. It contains a list of functions on the left: 'Variables and Parameters' (selected), 'Columns', 'Mathematical Functions', 'String Functions', 'Date/Time Functions', 'NULL Functions', 'Type Casts', and 'Operators'. The 'Description' field is empty.

Derived Column Name	Derived Column	Expression	Data Type
end_date	<add as new column>	(DT_DBDATE)(@[System::StartTime])	database date [DT_D...

At the bottom of the dialog, there are three buttons: 'Configure Error Output...', 'OK', 'Cancel', and 'Help'.

Derived Column 2 (start_date): This component assigns the current system date as the starting validity date for any newly inserted records or updated versions of existing records.



5.4 Error Handling & Validation

- Schema Synchronization: Validation errors such as Invalid object name or Metadata out of sync were resolved by ensuring the SQL Server tables were correctly defined and refreshing the SSIS metadata mappings.
- Staging Maintenance: To ensure data freshness, the ETL workflow includes a Truncate Staging task at the beginning of each cycle, preventing duplicate or stale data from polluting the current run.

Task 6: ETL Development – Accumulating Fact Tables

1. Fact Table Extension

The Fact_Admission_Billing table was structurally modified to support accumulating duration metrics by adding the following specific columns:

- `accm_txn_create_time` (datetime): Captures the exact system timestamp when the record is initially inserted into the Data Warehouse.
- `accm_txn_complete_time` (datetime): Captures the subsequent timestamp when the transaction (e.g., the discharge or final billing) is fully completed.
- `txn_process_time_hours` (decimal): A calculated metric storing the time difference in hours between the creation and completion of the transaction.

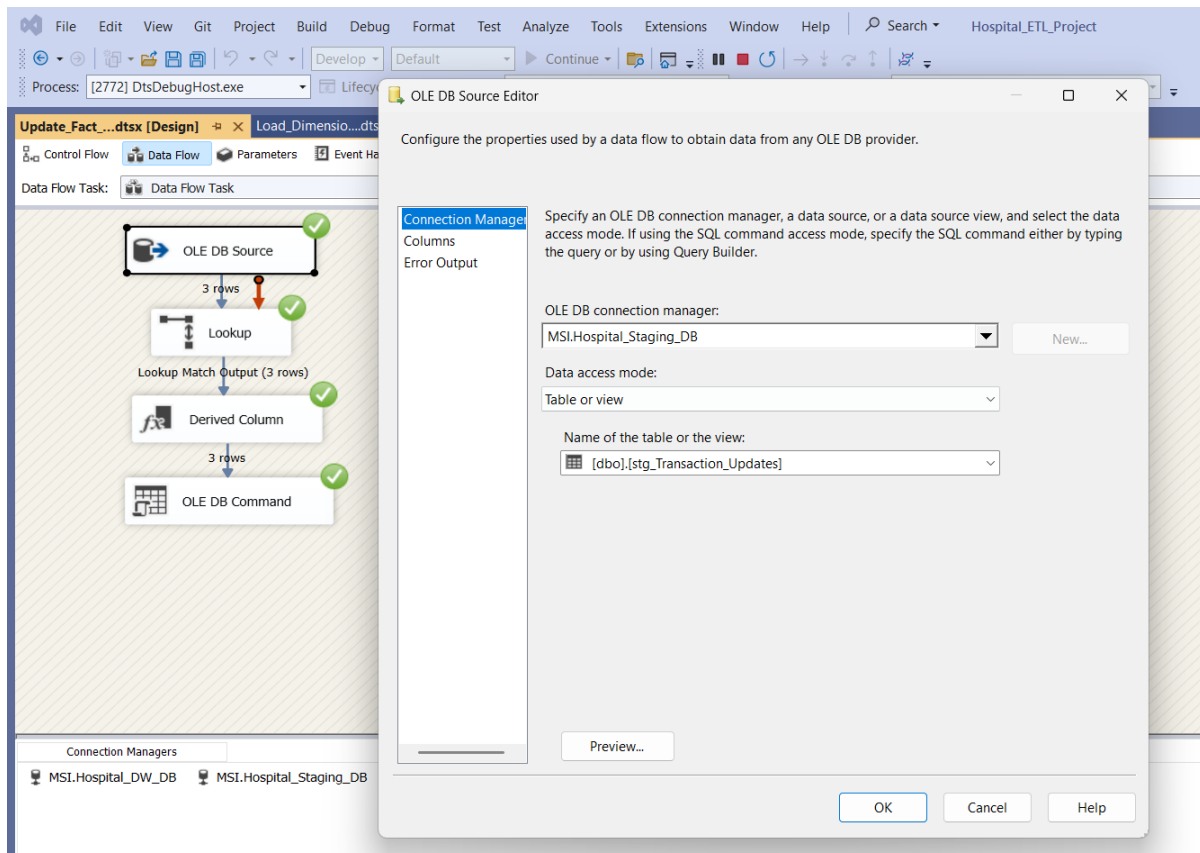
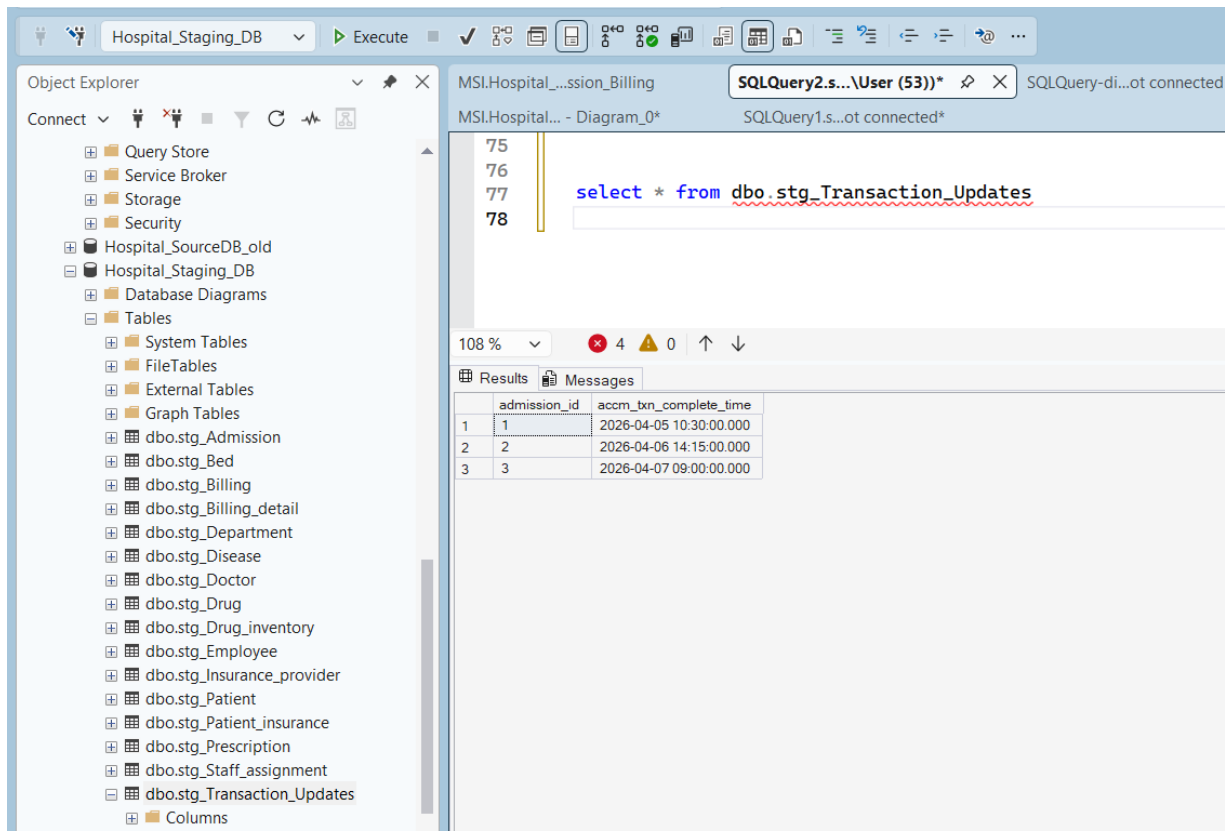
2. Initial Load Configuration

During the primary execution of the Fact table load, the `accm_txn_create_time` is populated dynamically using the current system timestamp. At this initial stage, both `accm_txn_complete_time` and `txn_process_time_hours` remain NULL as the transactions are inherently incomplete.

3. Implementation Logic for Accumulating Updates

To handle the completion of these transactions at a later date, a dedicated SSIS package was designed to perform the accumulating updates. The data flow strictly follows this sequence:

- **Data Source Extraction:** An OLE DB Source reads from a specifically prepared staging dataset (`dbo.stg_Transaction_Updates`) which contains the natural key (`admission_id`) and the newly recorded `accm_txn_complete_time`.
- **Lookup Transformation:** A Lookup connects to the existing Fact_Admission_Billing table. It performs an equi-join using the `admission_id` to retrieve the previously recorded `accm_txn_create_time` for that specific transaction.
- **Derived Column Calculation:** With both timestamps now available in the data pipeline, an SSIS expression utilizes the DATEDIFF function (`DATEDIFF("hh", accm_txn_create_time, accm_txn_complete_time)`) to dynamically calculate the total processing duration in hours.
- **OLE DB Command Update:** Finally, an OLE DB Command executes a parameterized SQL UPDATE statement row-by-row. It seamlessly maps the incoming pipeline parameters to accurately update the `accm_txn_complete_time` and `txn_process_time_hours` within the Fact table where the `admission_id` matches, completing the accumulating snapshot process.



File Edit View Git Project Build Debug Format Test Analyze Tools Extensions Window Help Hospital_ETL_Project

Process: [2772] DtsDebugHost.exe

Update_Fact_...dtsx [Design] Load_Dimension...

Control Flow Data Flow Parameters Event Task

Data Flow Task: Data Flow Task

OLE DB Source

3 rows

Lookup

Lookup Match Output (3 rows)

Derived Column

3 rows

OLE DB Command

Connection Managers

MSI.Hospital_DW_DB MSI.Hospital_Staging_DB

Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

Variables and Parameters Columns Mathematical Functions String Functions Date/Time Functions NULL Functions Type Casts Operators

Description:

Derived Column Name	Derived Column	Expression	Data Type	Update Existing
calculated_hours	<add as new column>	DATEDIFF("hh",accm_txn_create_time,accm_txn...	four-byte signed inte...	

Configure Error Output... OK Cancel Help

72
73
74
75
76

select * from Fact_Admission_Billing

108 %

4

</

Sri Lanka Institute of Information Technology



Data Warehousing and Business Intelligence (IT3021) Assignment 2

IT Number	Name	Email	Batch	Center
IT23343320	Senadeera D.M.K.K	IT23343320@my.sliit.lk	Y3.S2.WE.DS.02.02	Malabe

Contents

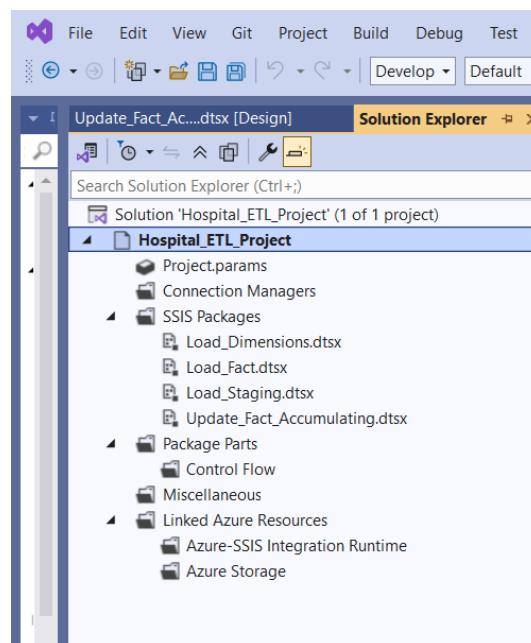
1. Data source for the Assignment 2	3
1.1 Description of the Data Source (Data Warehouse)	3
1.2 ETL Process Overview	3
1.3 Key Components of the Dimensional Model.....	4
1.4 Entity-Relationship (ER) Diagram.....	5
2. SSAS Cube implementation	6
2.1 Project Structure and Data Source Setup	6
2.2 Creating the Data Source View (DSV)	7
2.3 Designing the Cube and Defining Measures	8
2.4 Implementing Hierarchies	8
3. Demonstration of OLAP operations.....	10
3.1 Drill-Down.....	10
3.2. Roll-Up (Aggregation).....	11
3.3. Slice.....	11
3.4. Dice	12
3.5 Pivot (Rotate)	12
4. PowerBI Reports	13
4.1 Data Preparation and Modeling	13
4.2 Report 1: Matrix Visual with Groupings.....	13
4.3 Report 2: Cascading Slicers and Multiple Visuals	14
4.4 Report 3: Drill-Down Hierarchical Report.....	14
4.5 Report 4: Drill-Through Report.....	16
4.6 Publishing to Power BI Service	17

1. Data source for the Assignment 2

1.1 Description of the Data Source (Data Warehouse)

The primary data source for Assignment 2 is the Enterprise Data Warehouse (EDW) designed and populated during Assignment 1. The underlying source data is derived from a comprehensive, highly relational **Hospital Management Information System (HMIS)** dataset representing healthcare analytics, encompassing operational insights such as patient admissions, disease diagnostics, pharmacy inventory, and billing.

The analytical database, named **Hospital_DW_DB**, was designed following a standard **Star Schema** dimensional model to facilitate efficient Online Analytical Processing (OLAP) and Business Intelligence (BI) reporting.



1.2 ETL Process Overview

To construct this Data Warehouse, a robust ETL (Extract, Transform, Load) pipeline was developed using **SQL Server Integration Services (SSIS)** (Project: Hospital_ETL_Project).

- **Extraction:** Data was extracted from the normalized transactional database (Hospital_Source_DB) into a dedicated Staging database (Hospital_Staging_DB) via the Load_Staging.dtsx package.
- **Transformation & Loading:** The Load_Dimensions.dtsx package applied necessary transformations, including Slowly Changing Dimensions (SCD), to load data into the dimension tables. Subsequently, the Load_Fact.dtsx package aggregated and loaded the transactional data into the central fact table.

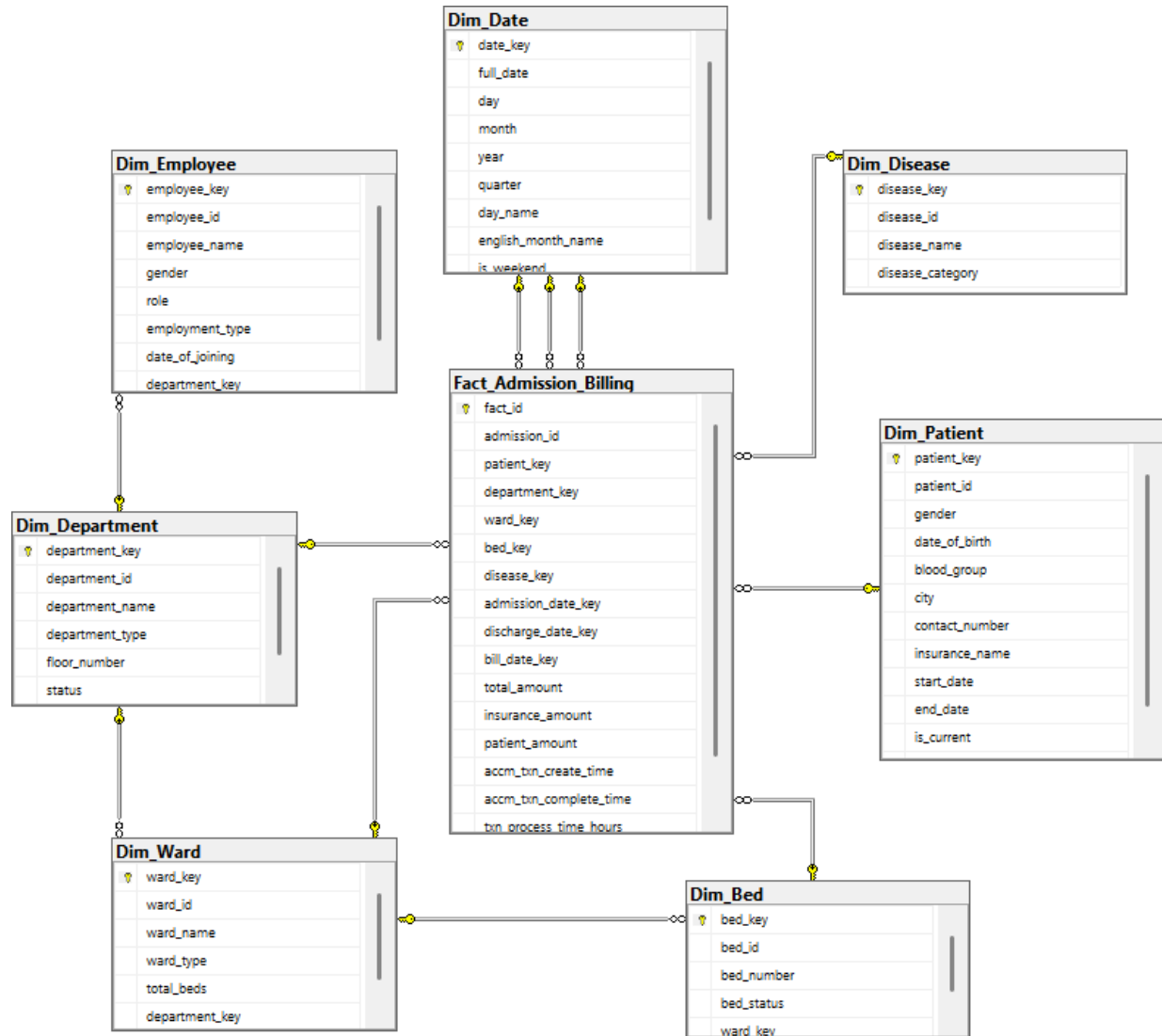
1.3 Key Components of the Dimensional Model

The Star Schema architecture consists of the following key tables designed to analyze hospital revenue and patient admissions:

- **Fact Table:**
 - **Fact_Admission_Billing:** The central fact table storing quantitative metrics such as Total Amount (Revenue) and Patient Admission Counts.
- **Dimension Tables:**
 - **Dim_Patient:** Demographics and details of patients.
 - **Dim_Disease:** Medical conditions categorized by disease type.
 - **Dim_Ward & Dim_Department:** Location and departmental hierarchy details.
 - **Dim_Drug:** Pharmacy and medication details.
 - **Dim_Employee:** Staff and administrative details.
 - **Dim_Date:** Time dimension for hierarchical temporal analysis.

1.4 Entity-Relationship (ER) Diagram

The following diagram illustrates the Star Schema design of the Hospital_DW_DB, showcasing the central fact table surrounded by its associated dimension tables.

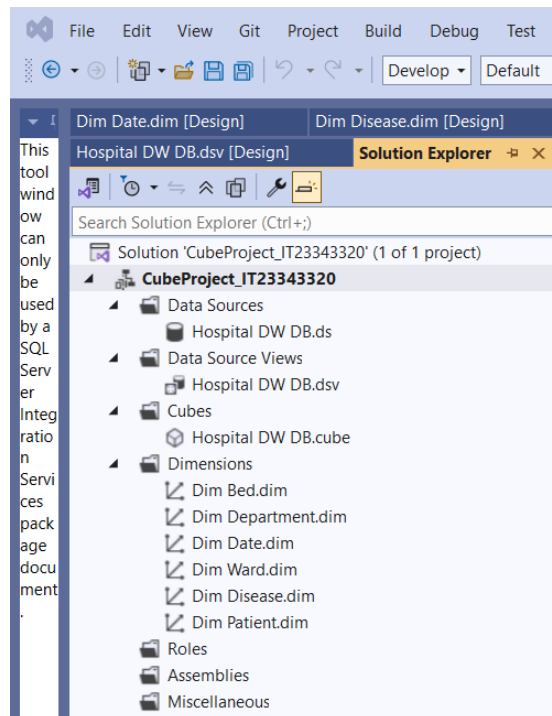


2. SSAS Cube implementation

The multidimensional SSAS Cube was implemented using SQL Server Data Tools (SSDT) in Visual Studio to enable fast, aggregated analysis of the hospital data. The process involved defining the Data Source, creating a Data Source View (DSV), and designing the final Cube with dimensions, measures, and hierarchies.

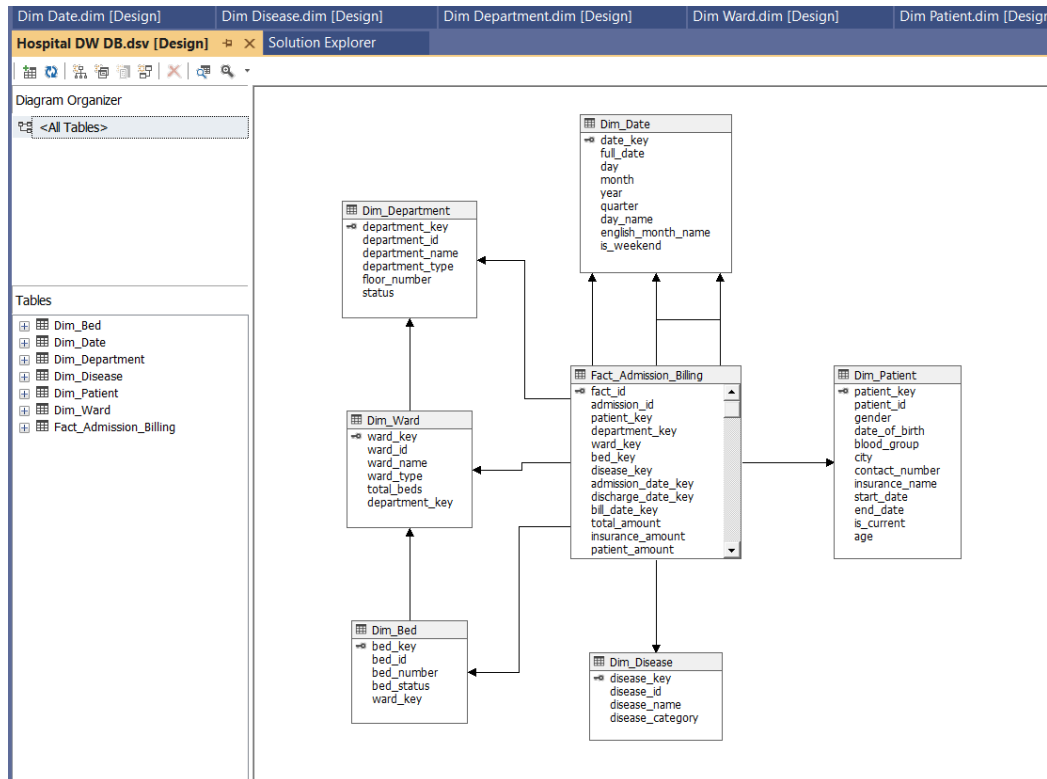
2.1 Project Structure and Data Source Setup

The project CubeProject_IT23343320 was created using the Analysis Services Multidimensional and Data Mining Project template. A connection was established to the underlying Data Warehouse (Hospital_DW_DB) as the primary data source.



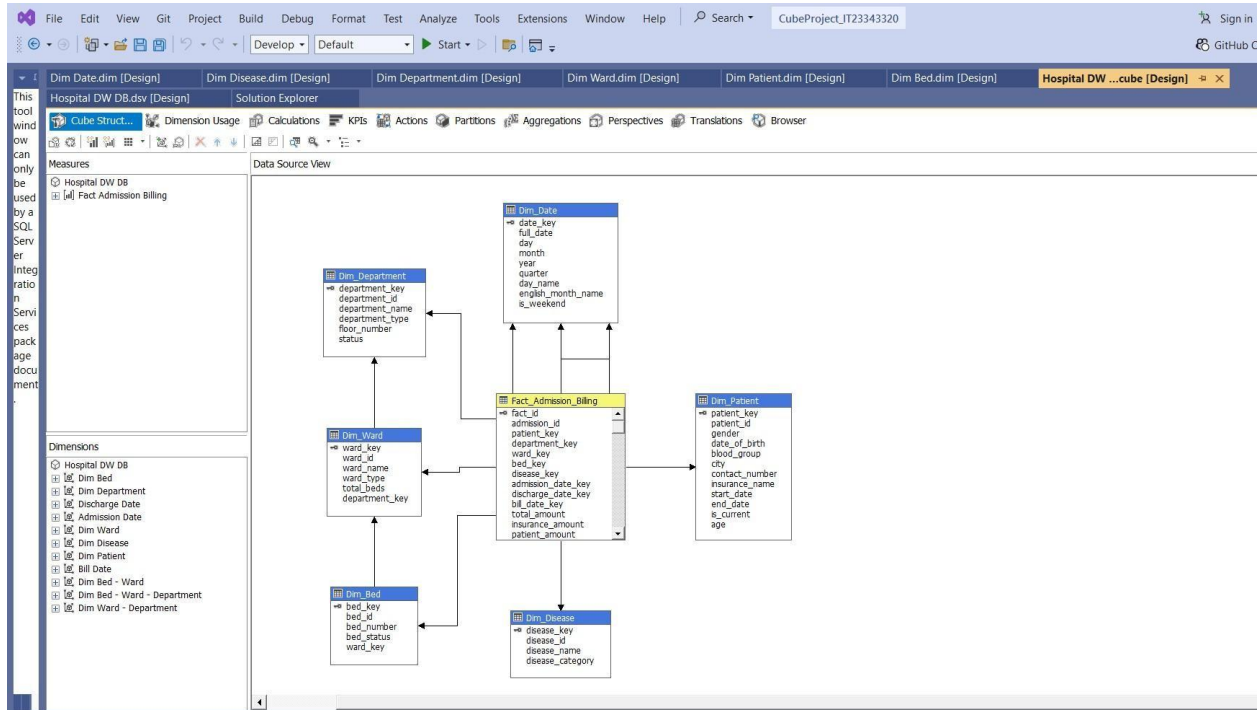
2.2 Creating the Data Source View (DSV)

A Data Source View was created to logically represent the Star Schema within the SSAS environment. The relationships between the central Fact_Admission_Billing table and its surrounding dimensions (Dim_Patient, Dim_Disease, Dim_Ward, Dim_Department, Dim_Bed, and Dim_Date) were defined here using primary and foreign key constraints.



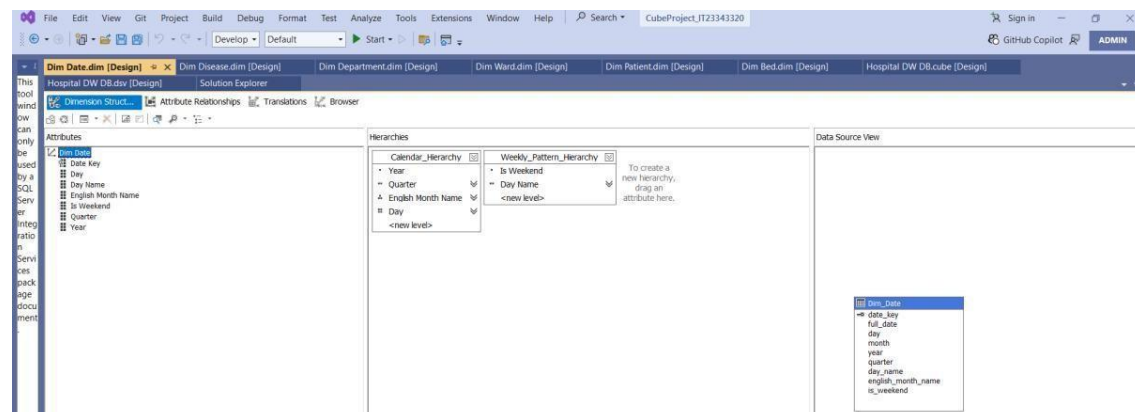
2.3 Designing the Cube and Defining Measures

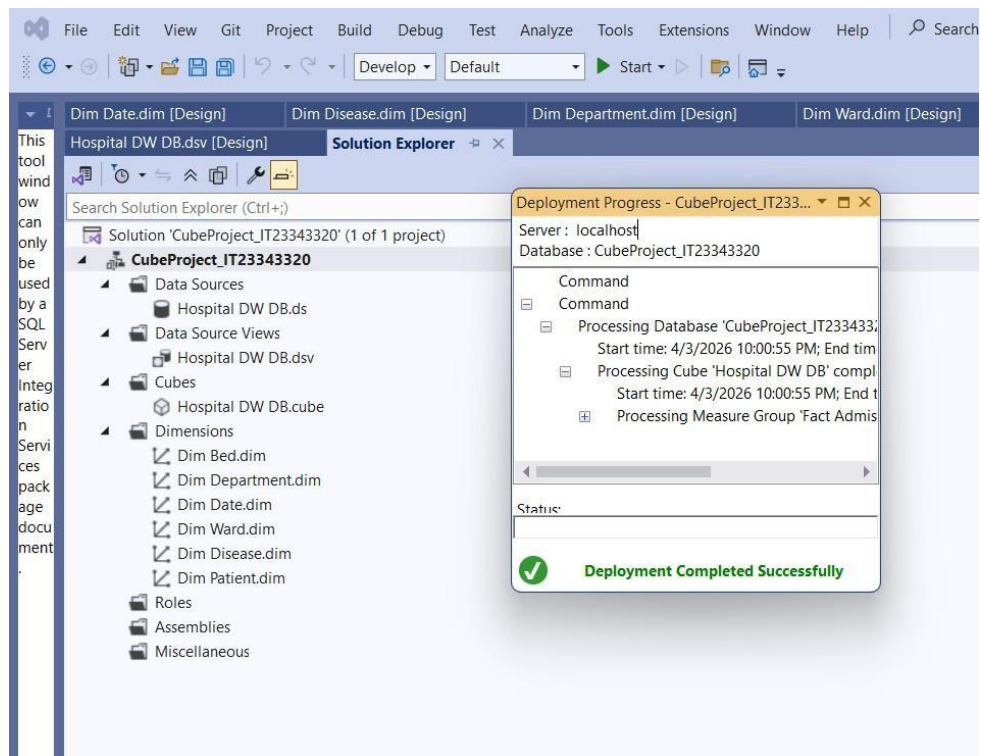
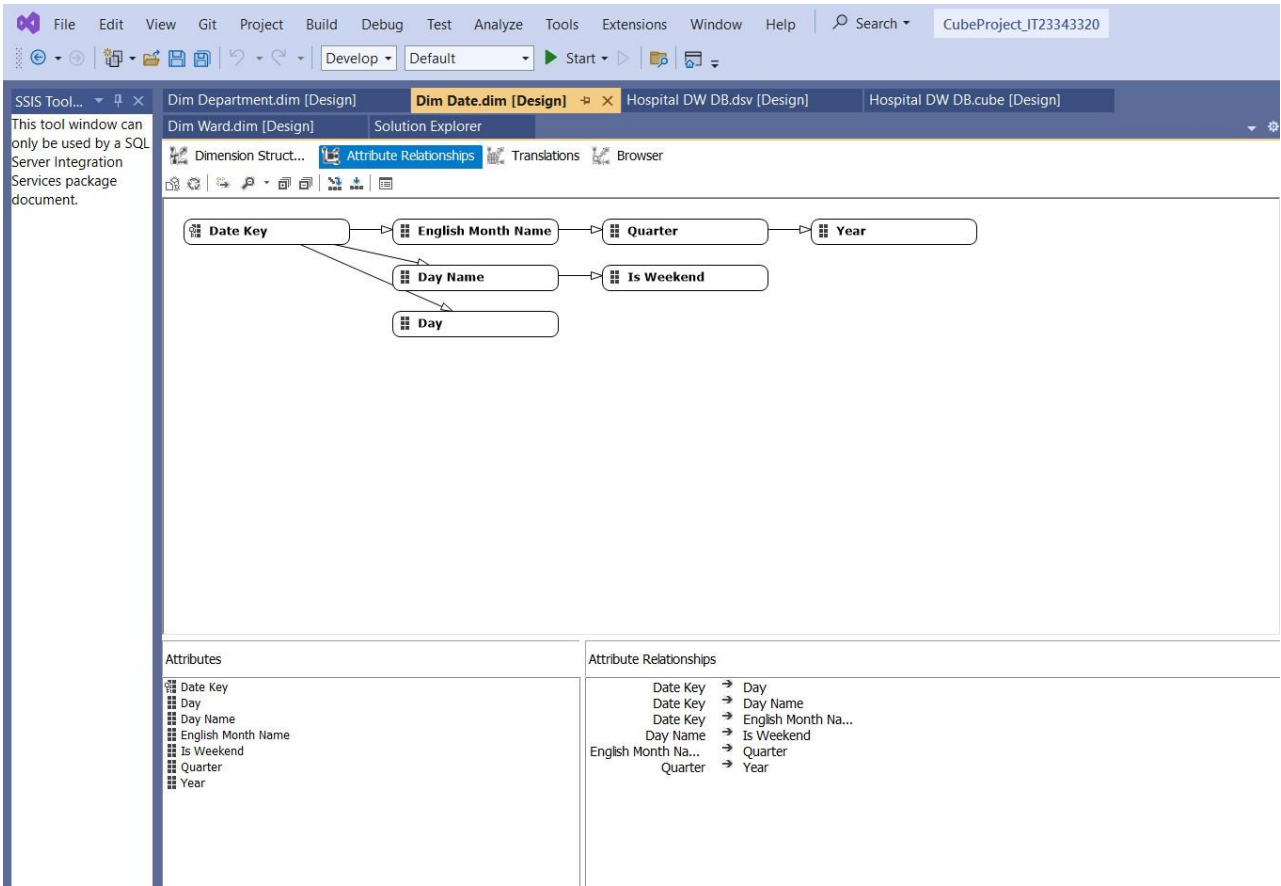
The Cube was generated based on the DSV. The Fact_Admission_Billing table was selected as the Measure Group. Relevant quantitative measures, such as Total Amount and Fact Admission Billing Count, were created with appropriate aggregation functions (e.g., Sum, Count) to analyze hospital revenue and patient volume.



2.4 Implementing Hierarchies

To enable Drill-down and Roll-up OLAP operations, multidimensional hierarchies were implemented within the dimensions. A Calendar_Hierarchy was successfully created within the Admission Date dimension, structuring the data logically from Year -> Quarter -> English Month Name -> Day. This allows users to drill down from annual revenue to daily revenue seamlessly.





3. Demonstration of OLAP operations

To demonstrate the analytical capabilities of the implemented SSAS Cube, an Microsoft Excel workbook was connected to the cube as a front-end analysis tool. Excel's Pivot Table and Pivot Chart features were utilized to perform various OLAP operations.

3.1 Drill-Down

Drill-down allows users to navigate from a higher level of summary data to a more detailed level.

- **Action:** In the Pivot Table, expand the Year attribute in the Calendar_Hierarchy to see the Quarter and Month level revenue.

The screenshot displays the Microsoft Excel interface with a PivotTable and the PivotTable Fields task pane. The PivotTable is filtered by Gender (Female) and shows revenue data for the years 2023, 2024, and 2025, broken down by month. The PivotTable Fields task pane on the right shows the hierarchy for the PivotTable, with 'Admission Date.Calendar_Hierarchy' expanded to show the 'Month' level.

Row Labels	Total Amount
2023	
December	51252990
November	50737666
October	49865152
2024	
April	154359014
June	51963590
May	143170190
2025	
February	50075832
January	101072138
March	140011386
2028	343360982
Grand Total	1507867372

PivotTable Fields

Choose fields to add to report:

Search

Admission Date
Admission Date.Calendar_Hierarchy
Admission Date.Weekly_Pattern_Hierarchy
More Fields

Drag fields between areas below:

Filters: Gender

Columns: Admission Date.Calendar_Hierarchy

Values: Total Amount

Defer Layout Update

3.2. Roll-Up (Aggregation)

Roll-up is the opposite of drill-down, where data is summarized by climbing up the hierarchy.

- **Action:** Collapse of the Month and Quarter levels to show the total revenue aggregated at the Year level.

The screenshot shows the Microsoft Excel interface. The PivotTable is located in the range A3:B8. The PivotTable Fields task pane is open on the right, showing the following configuration:

- Filters:** Gender
- Rows:** Admission Date.Calendar_Hierarchy
- Values:** Total Amount

The PivotTable data is as follows:

Gender	Total Amount
Female	1507867372

3.3. Slice

Slicing performs a sub-setting operation by picking a specific value for one of its dimensions.

- **Action:** Use a Slicer to filter the data for a specific **Gender** (e.g., 'Female') from the Dim_Patient dimension.

	A	B	C	D	E	F
1						
2						
3	Row Labels	Total Amount				
4	* 2023	523854240				
5	* 2024	349492794				
6	* 2025	291159356				
7	* 2028	343360982				
8	Grand Total	1507867372				
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						

	A	B	C	D	E	F
1						
2						
3	Row Labels	Total Amount				
4	* 2023	621924326				
5	* 2024	420067590				
6	* 2025	345314226				
7	* 2028	405309812				
8	Grand Total	1792615954				
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						

	A	B	C	D	E	F
1						
2						
3	Row Labels	Total Amount				
4	* 2023	24130284				
5	* 2024	17716266				
6	* 2025	12409996				
7	* 2028	13752346				
8	Grand Total	68008892				
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						

3.4. Dice

Dicing defines a sub-cube by selecting specific values for two or more dimensions.

- **Action:** Filter the data by selecting a specific **Gender** (e.g., 'Male') AND a specific **Disease Category** (e.g., 'Chronic') using two slicers.

Row Labels	Total Amount
2023	65551314
2024	44074844
2025	31683506
2028	42740652
Grand Total	184050316

Gender Slicer: Female, Male (selected), Other

Disease Category Slicer: Cardiac (selected), Endocrine, Hematological, Infectious, Neurological, Orthopedic, Pediatric, Renal

3.5 Pivot (Rotate)

Pivot is a visualization operation that rotates the data axes to provide an alternative view of the data.

- **Action:** Swap the positions of Dim_Patient (Gender) from Columns to Rows and Dim_Date (Year) from Rows to Columns.

Row Labels	Total Amount
2023	
4 December	
15	6022934
November	
11	7008288
23	6283060
October	46237032
2024	
2 April	
April	17346920
June	6512584
May	20215340
2025	
1 February	
February	5822444
January	11432648
March	14428414
2028	42740652
Grand Total	184050316

Gender	Male
2023	65551314
2024	44074844
2025	31683506
2028	42740652
Grand Total	184050316

After Pivot: Gender in Rows, Year in Columns

Before Pivot: Year in Rows, Gender in Columns

4. PowerBI Reports

4.1 Data Preparation and Modeling

To develop Power BI reports, a **Live Connection** was established to the previously built SQL Server Analysis Services (SSAS) Cube (CubeProject_IT23343320). Since a Live Connection was utilized, the data modeling, DAX measure calculations (e.g., Total Amount, Admission Count), and hierarchies (e.g., Calendar_Hierarchy) were seamlessly inherited from the SSAS cube, ensuring centralized business logic and optimized query performance.

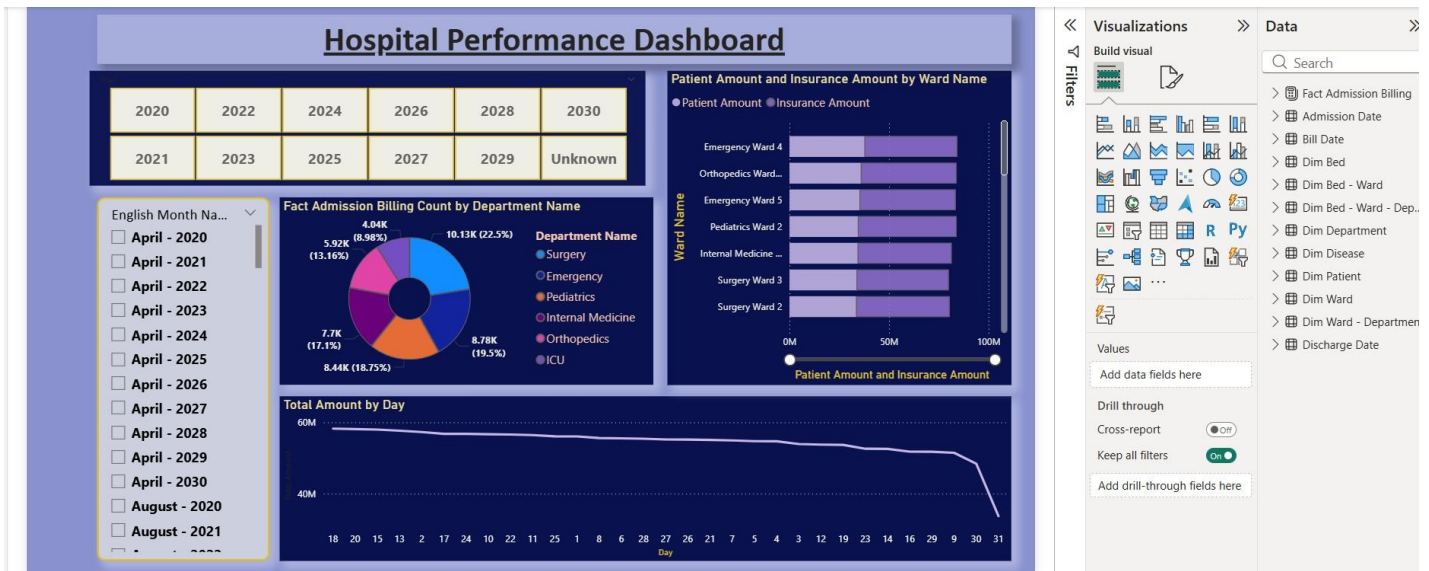
4.2 Report 1: Matrix Visual with Groupings

- **Objective:** To display detailed tabular revenue and admission data with row and column groupings.



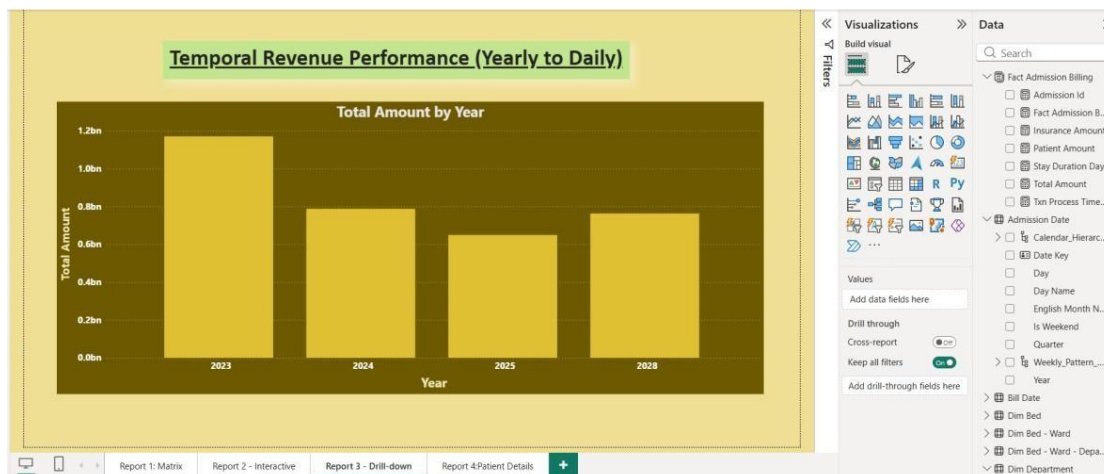
4.3 Report 2: Cascading Slicers and Multiple Visuals

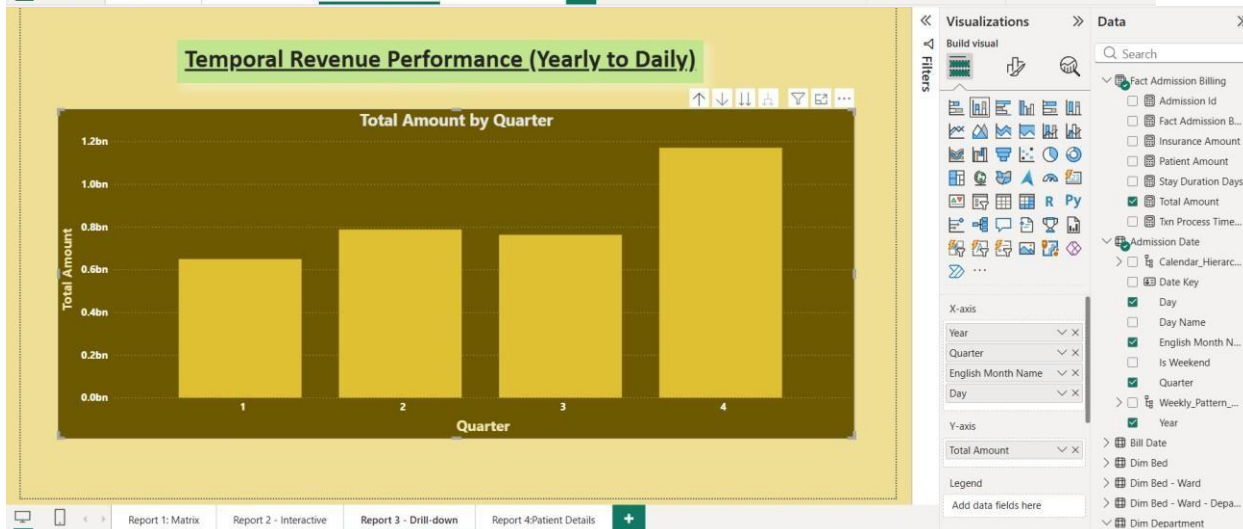
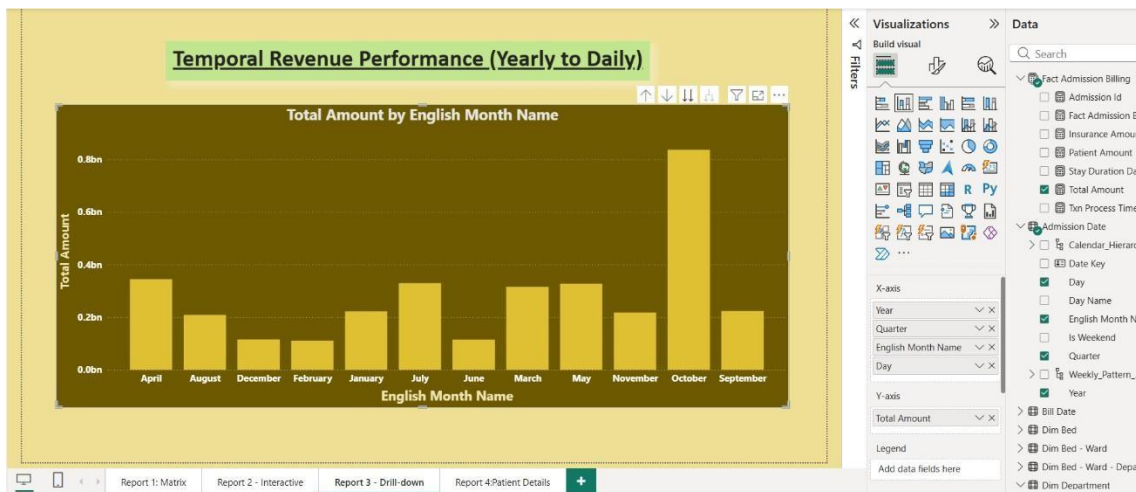
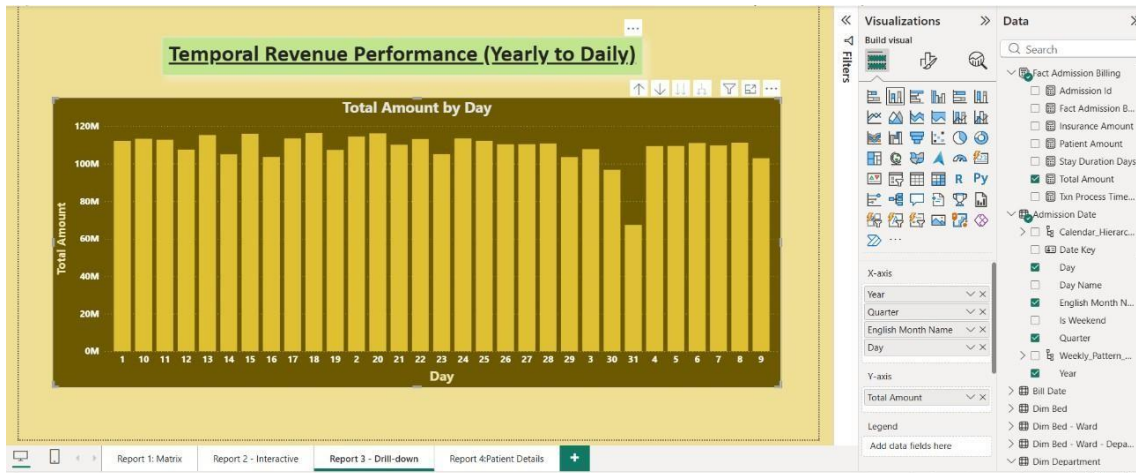
- **Objective:** To implement dynamically filtering (cascading) slicers and present insights using multiple charts.



4.4 Report 3: Drill-Down Hierarchical Report

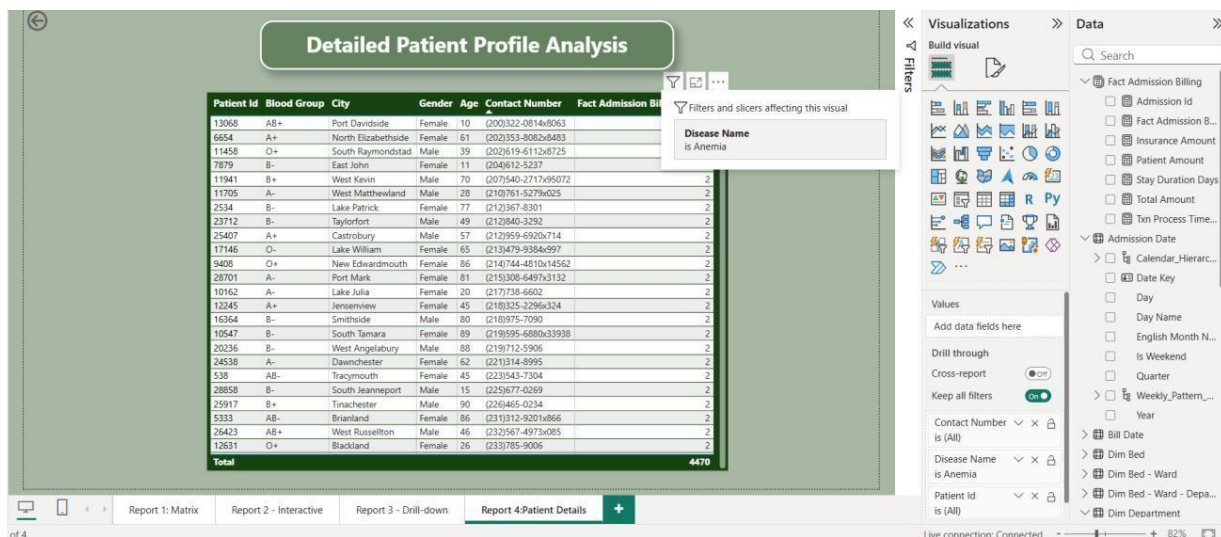
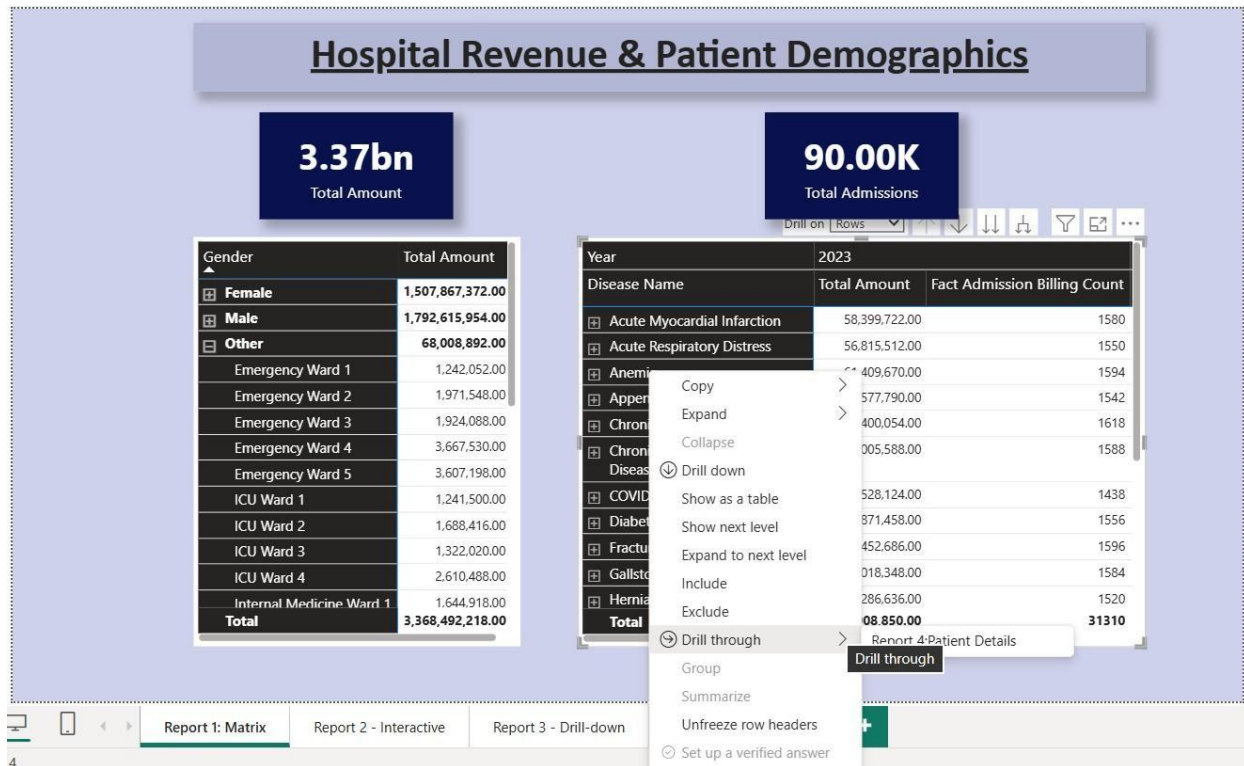
- **Objective:** To allow users to explore revenue data hierarchically across time periods.





4.5 Report 4: Drill-Through Report

- **Objective:** To allow users to right-click a summarized data point on a main page and navigate to a detailed page.



4.6 Publishing to Power BI Service

After developing the reports locally, the .pbix file was successfully published to the Power BI Service workspace for online access and sharing. (Note: Due to the use of an Enterprise SSAS Live Connection, an On-Premises Standard Data Gateway is required to render live visuals in the cloud, hence the visualizations were primarily validated in Power BI Desktop).

